ARCnordic Tid

# Kodeoversigt og funktioner

Arkitektur, GRASP og OOP, moduler og funktioner, datamodel, sikkerhed, test og vedligeholdelse

| 23.647 | 388 | 205 | 103 |
| --- | --- | --- | --- |
| linjer kildekode | dokumenterede funktioner/typer | automatiske tests | API-endpoints |

Udarbejdet 9. november 2026 · version 1bda6a3 (28 commits) · github.com/Deb2121/ARC-Time
Genereres fra koden med `node scripts/code-report.mjs` — inventarlisterne passer altid med koden.

# Indhold


1. Resumé — hvad systemet er, og hvor det står
2. Arkitektur — lag, pakker og en request fra start til slut
3. GRASP og OOP i koden
4. Clean code — konventioner der holder koden vedligeholdelig
5. Sikkerhed og adgangsstyring (rollematrix)
6. Data og privatliv (GDPR)
7. Test og kvalitetssikring
8. Vedligeholdelse — sådan tilføjer man en funktion
9. Moduler og funktioner (inventar)
10. API-endpoints
11. Datamodel (tabeller)
12. Beslutningslog
13. Kendte begrænsninger og anbefalinger


# 1. Resumé

**ARCnordic Tid** er et PSA-system (tid, projekter, budget, godkendelse, fakturering, udlæg, fravær og flex, ressourceplanlægning, løneksport og rapportering) bygget som en ren genimplementering. Systemet er bygget i faser: M0 fundament, M1 tidsregistrering og godkendelse, M2 fakturering og perioder, M3 drift (udlæg, ressourceplan, dashboard, Outlook, analyse), M4 løneksport og import, samt rettelser efter kodegennemgang (adgangspolitik, privatliv, revisionsspor, GDPR, backup).

| | | | |
|---|---|---|---|
| 1.895<br>packages/domain · 35 filer | 3.823<br>packages/db · 15 filer | 297<br>packages/integrations · 6 filer | 130<br>packages/ui · 1 filer |
| 7.644<br>apps/api · 54 filer | 9.419<br>apps/web · 44 filer | 439<br>apps/analytics · 9 filer | |

**Teknologi:** TypeScript overalt (Node 26, pnpm-monorepo), Next.js 16 og React 19 til web, Fastify 5 og Zod til API, Drizzle ORM og PostgreSQL 17 med row-level security, Python 3.12 og FastAPI til analyse. Biome (lint og format), Vitest, pytest og Playwright til test. Alt kører lokalt i Docker (D-017); driftsforslag ligger i `docs/operations.md`.

**Designprincipper:** forretningsregler er rene funktioner i domænelaget; alle eksterne systemer sidder bag adaptere og er slået fra som standard; al adgangslogik bor ét sted (`AccessPolicy`); hver fil har en forklarende header og hver eksporteret funktion en JSDoc/docstring, håndhævet automatisk.

## 2. Arkitektur

Koden er et monorepo med tre apps og fire pakker. Afhængigheder går kun nedad i diagrammet: domænet kender ingen andre lag, og web-appen taler kun med API'et via HTTP.

**apps/web — Next.js (brugerflade)**
Sider pr. skærm (`app/(app)/…`), genbrugelige komponenter, i18n (dansk/engelsk). Viser kun handlinger, som serverens rollematrix tillader (kapabiliteter fra `/me`).

HTTP + session-cookie + CSRF-header

**apps/api — Fastify (GRASP Controller + services)**
`routes/`: validér (Zod) → tjek adgang (`requireCapability`) → kald service → svar. `services/`: arbejdsgange (godkendelse, fakturaer, løn, GDPR …). Baggrundsjob: outbox, påmindelser, opbevaringsfrist.

funktionskald

**packages/domain — forretningsregler (Information Expert)**
Rene funktioner: normtid, flex, ferie, satser, faktureringsforslag, nøgletal, løn, import, GDPR og `AccessPolicy`. Ingen I/O → hurtige, sikre tests.

**packages/db — Drizzle + PostgreSQL**
`schema.ts` (alle tabeller), migreringer, RLS pr. virksomhed via `withTenant()`, audit-skriver, transaktionelle arbejdsgange (bogføring, perioder).

**packages/integrations — adaptere (Polymorphism / Protected Variations)**
e-conomic (regnskab) og Microsoft Graph (kalender). Interfaces gør dem udskiftelige; slået fra uden konfiguration.

**apps/analytics — Python FastAPI (skrivebeskyttet)**
Nøgletal, Excel og Power BI-datasæt via en databaserolle med kun SELECT. Formlerne spejler `kpi.ts` og testes med samme eksempler.

### En request fra start til slut — "gem en celle i ugesedlen"

1. Brugeren skriver 7,5 i en celle → `week-grid.tsx` sender `PUT /api/v1/timesheet/entries` med CSRF-header.
2. `auth/plugin.ts` slår sessionen op (kun hash gemmes), indlæser roller og ekstra rettigheder og afviser manglende CSRF.
3. `routes/timesheet.ts` validerer body med Zod og kalder `saveCell()` i `services/timesheet.ts`.
4. Servicen henter data i en transaktion med `app.tenant_id` sat (RLS), og kalder domænets `validateEntry()`: 24-timers-grænse, 0,25-trin, lukket periode, påkrævet kommentar.
5. Gemmer rækken; projektleder-stemplet nulstilles, hvis rækken ændres. Fejl kastes som `AppError` og oversættes centralt til `{ error: kode }`, som web-appen viser på dansk.

# 3. GRASP og OOP i koden

| Princip | Hvor det ses |
|---|---|
| **Information Expert** | Beregninger bor hos de data, de kræver: `norm.ts`, `flex.ts`, `rates.ts`, `invoicing.ts`, `payroll.ts`, `kpi.ts`; `AccessPolicy` ved, hvem der må hvad; `people.ts` ved, hvem der er leder for hvem. |
| **Creator** | Den, der ejer noget, opretter det: en uge opretter sin ugeseddel-periode (`ensurePeriod`), et projekt opretter sin første kontrakt og PL-medlemskab, en faktura sine linjer og kildereferencer. |
| **Controller** | Kun rutefilerne: validér → autorisér → delegér → svar. Ingen SQL-arbejdsgange eller regler i ruterne. |
| **Low Coupling** | Services kaster `AppError` og kender ikke HTTP. Domænet har ingen afhængigheder. Web kender kun API'ets JSON. Eksterne systemer nås kun via adaptere. |
| **High Cohesion** | Ét ansvar pr. modul: `project-setup.ts` ændrer projekter, `projects.ts` læser dem; `audit-log.ts` læser sporet, `audit.ts` skriver det. |
| **Polymorphism** | `AccountingAdapter`, `CalendarSource`, `CalendarProvider`, `Mailer` — flere implementeringer bag samme interface (rigtige og falske i tests). |
| **Pure Fabrication** | Klasser/moduler for god struktur: outbox, audit-skriver, rapport/CSV-bygger, løneksport, GDPR-service, Excel-bygger. |
| **Indirection** | `withTenant()` mellem kode og database; transaktionel outbox mellem bogføring og regnskabssystem; `/me`-kapabiliteter mellem server-regler og knapper i UI. |
| **Protected Variations** | RLS, triggers der gør bogførte fakturaer uforanderlige, append-only audit_log, adapter-interfaces, i18n-nøgler, rollematrix som data. |

### Objektorienteret, hvor det giver mening

Kernen er bevidst **funktionel** (rene funktioner over simple data er lettest at teste og at spejle i Python). Objekter bruges, hvor de modellerer noget med tilstand og adfærd, eller en udskiftelig samarbejdspartner:

| Klasse / interface | Hvorfor et objekt |
|---|---|
| `AccessPolicy` | Indkapsler én persons roller + rettigheder. Metoderne `can()`, `capabilities()`, `seesAbsenceReason()`, `canReadAudit()`. Matrixen er privat data — kaldere ser aldrig rollelister. |
| `EconomicAdapter`, `GraphCalendarSource` | Implementerer interfaces (polymorfi); kan udskiftes uden at røre kaldere; tests bruger falske udgaver. |
| `AppError, InvoiceError, PeriodError, IntegrationError` | Fejltyper bærer HTTP-status/"prøv igen"-flag; én central fejlhåndtering oversætter dem. |
| `Caller (Python dataclass)` | Uforanderligt værdiobjekt for den indloggede i analysetjenesten. |

Eksempel — hele rollematrixen spørges gennem én metode:

```
/** True when the matrix allows the action for this person. */
  can(capability: Capability): boolean {
    const rule = MATRIX[capability];
    if (hasRole(this.roles, ...rule.roles)) return true;
    return Boolean(
      rule.right && this.rights.includes(rule.right.right) && hasRole(this.roles, ...rule.right.rightFor),
    );
  }
```

## 4. Clean code — konventioner

- **Én sandhed pr. regel.** Rollematrix (`access.ts`), beløbslinjer (`isAmountLine`), nøgletal (`kpi.ts` + Python-spejl testet med samme eksempler).
- **Enheder er heltal.** Penge i øre, timer i centitimer (7,5 t = 750), procent i basispoint (81,08 % = 8108). Ingen kommatal i beregninger → ingen afrundingsfejl. Datoer er lokale ISO-datoer i Europe/Copenhagen.
- **Dokumentation håndhæves.** `scripts/check-docs.mjs` kræver header i hver fil og JSDoc/docstring på hver eksport (TypeScript og Python) — del af `pnpm check`, pre-commit-hook og CI.
- **Navne fortæller hvad.** Funktioner er verber (`saveCell`, `exportPayroll`), regler er navneord (`retentionCutoff`). Fejlkoder er maskinlæsbare (`period_closed`) og oversættes i UI.
- **Små, tynde lag.** Ruter er korte; arbejdsgange i services; regler i domænet. Ingen forretningslogik i React-komponenter ud over visning.
- **Sikker som standard.** Integrationer, mail og kalender er slået fra uden konfiguration; dev-login kan aldrig være slået til i produktion.
- **Automatisk formattering og lint** (Biome) — ingen stilkampe i code review.

## 5. Sikkerhed og adgangsstyring

- **Login:** uigennemsigtige session-tokens i httpOnly-cookie (kun SHA-256-hash gemmes), CSRF-token på alle ændringer, rate limit på login. Microsoft Entra ID (OIDC + PKCE) er bygget, men slået fra lokalt.
- **Virksomhedsadskillelse:** PostgreSQL row-level security med en ikke-ejer app-rolle; hver forespørgsel kører med `app.tenant_id` sat. Analysetjenesten har sin egen rolle med kun SELECT.
- **Uforanderlighed:** bogførte fakturaer er låst af database-triggers; revisionssporet (`audit_log`) kan kun indsættes i — app-rollen har ikke UPDATE/DELETE.
- **Funktionsadskillelse (D-025):** Admin konfigurerer, Økonomi bogfører, krediterer, lukker perioder og eksporterer løn.
- **Privatliv (OQ-21):** sygdom vises som "Fravær" uden kommentar for alle andre end medarbejderen, lederen og Økonomi — i ugevisning, rapporter og analyse.

### Rollematrix (genereret fra `access.ts`)

| Handling | Medarbejder | Projektleder | Leder | Økonomi | Ledelse | Admin |
|---|---|---|---|---|---|---|
| `project.create` | – | ◐ | – | ✓ | – | ✓ |
| `invoice.prepare` | – | ✓ | – | ✓ | – | – |
| `invoice.prepare_all` | – | – | – | ✓ | – | – |
| `invoice.book` | – | – | – | ✓ | – | – |
| `invoice.credit` | – | – | – | ✓ | – | – |
| `period.view` | – | – | – | ✓ | ✓ | ✓ |
| `period.close` | – | – | – | ✓ | – | – |
| `payroll.export` | – | – | – | ✓ | – | – |
| `payroll.setup` | – | – | – | ✓ | – | ✓ |
| `expense.approve_any` | – | – | – | ✓ | – | – |
| `timesheet.approve_unmanaged` | – | – | – | ✓ | – | – |
| `audit.read` | – | – | – | ✓ | ✓ | ✓ |
| `audit.read_all` | – | – | – | – | ✓ | ✓ |
| `gdpr.manage` | – | – | – | – | – | ✓ |
| `settings.manage` | – | – | – | – | – | ✓ |
| `settings.read` | – | – | – | ✓ | – | ✓ |
| `pricelist.manage` | – | – | – | ✓ | – | ✓ |
| `people.import` | – | – | – | – | – | ✓ |
| `data.import` | – | – | – | ✓ | – | ✓ |
| `kpi.company` | – | – | – | ✓ | ✓ | ✓ |

◐ = kun med ekstra rettighed (fx `create_projects`, tildeles under Medarbejdere). Hvem der ses (data-omfang: egne, egne medarbejdere, egne projekter) afgøres i `services/people.ts` og `services/projects.ts`.

# 6. Data og privatliv (GDPR)

- **Dataudtræk:** Admin → Medarbejdere → "Hent alle persondata" giver én JSON-fil med alt om personen og intet om andre (`exportPersonalData`).
- **Sletning af fratrådte:** "Slet personoplysninger" pseudonymiserer navn, e-mail, initialer og lønnummer, fjerner kommentarer og sletter ikke-bogføringsdata. Bogføringsdata bevares i 5 år efter regnskabsåret. Blokeres, hvis personen er aktiv, projektleder på aktive projekter eller leder for andre (`erasePerson`, `erasureBlockers`).
- **Deaktivering** logger ud med det samme (sessioner og kalender-tokens slettes).
- **Natligt job** sletter udløbne sessioner/tokens og logs ældre end 13 måneder og rapporterer bogføringsdata ud over fristen (sletning af regnskabsdata er en bevidst handling efter aftale med revisor).
- **Fortegnelse:** `docs/gdpr/processing-inventory.md`.

### Drift og backup

Lokalt tages en backup dagligt ved opstart (14 gemmes) og ved nulstilling; gendannelse er verificeret. Forslag til produktion (Azure EU, administreret PostgreSQL med point-in-time restore, Key Vault, Blob Storage, Microsoft 365-mail) og flytning ud af OneDrive står i `docs/operations.md`.

# 7. Test og kvalitetssikring

| Område | Filer | Tests |
|---|---|---|
| Domæne | 11 | 87 |
| Database / RLS | 1 | 5 |
| Integrationer | 2 | 6 |
| API (integration mod rigtig database) | 11 | 75 |
| Python-analyse | 3 | 12 |
| E2E i browser (Playwright) | 12 | 20 |
| **I alt** | | **205** |

- **Domænetests** bruger de regneeksempler, der står i specifikationen (fx flex +0,5; 49 % færdig → 117.600 kr; 3,00 feriedage).
- **API-tests** kører mod den rigtige database med RLS. Destruktive forløb (låsning af lønperioder, import, GDPR-sletning) kører i en transaktion, der rulles tilbage — demodata røres ikke.
- **E2E-tests** kører i en rigtig browser på dansk mod produktionsbygningen i CI.
- **Kvalitetsport:** `pnpm check` = lint + dokumentationstjek + typecheck + alle tests. Pre-commit-hook kører lint, docs og typecheck; GitHub Actions kører det hele inkl. migrering, drift-tjek og E2E.

## 8. Vedligeholdelse — sådan tilføjer man en funktion

1. **Regel først:** skriv beregningen som ren funktion i `packages/domain/src/<emne>.ts` med JSDoc og et eksempel, og en test med tal fra specifikationen.
2. **Data:** tilføj tabel/kolonne i `packages/db/src/schema.ts` → `drizzle-kit generate`; nye tabeller med `tenant_id` får RLS via skabelon-migreringen (RLS-testen fejler ellers).
3. **Arbejdsgang:** service-funktion i `apps/api/src/services/`; skriv til revisionssporet ved ændringer af økonomiske data.
4. **Adgang:** tilføj en kapabilitet i `AccessPolicy`, hvis handlingen er ny; brug `requireCapability` i ruten og `can(me, …)` i web.
5. **Rute:** tynd Controller med Zod-skema i `apps/api/src/routes/`; registrér i `app.ts`.
6. **UI:** side under `apps/web/src/app/(app)/`, tekster i `i18n/da.ts` og `en.ts`.
7. **Test og dokumentation:** API-test (rul tilbage, hvis den skriver), evt. E2E; beslutning i `docs/decisions.md`; kør `pnpm check`; generér denne rapport igen.

**Hvor finder jeg …?** Arkitekturguiden `docs/architecture.md` (kodekort, GRASP, enheder), specifikationen `docs/spec/*.md`, beslutninger `docs/decisions.md`, åbne spørgsmål `docs/open-questions.md`, drift `docs/operations.md`, demo-gennemgange `docs/demo/`.

# 9. Moduler og funktioner

Udtrukket automatisk fra kildekoden: filens formål (header) og hver eksporteret funktion/type med første sætning af dens dokumentation. Sidefiler (Next.js-sider) er ikke med — deres formål er deres skærm. Beskrivelserne er på engelsk, fordi kodens kommentarer bevidst skrives på engelsk, så enhver udvikler kan læse dem; brugerfladen er på dansk og engelsk.

## Domæne (packages/domain) — forretningsregler

### packages/domain/src/absence-codes.ts

Default absence codes for a new Danish tenant (docs/spec/absence-flex.md).

| | |
|---|---|
| `AbsenceCodeDef` `interface` | Default absence codes for a new Danish tenant (docs/spec/absence-flex.md). |

### packages/domain/src/access.ts

Access policy (docs/spec/overview.md §3 role matrix, decision D-025) — GRASP Information Expert + Protected Variations: the one place that knows which role (or extra right) may do which action.

| | |
|---|---|
| `RIGHTS` `const` | Extra rights granted per person on top of roles (spec: "create if right"). |
| `CAPABILITIES` `const` | Actions guarded by the matrix. |
| `FINANCIAL_AUDIT_ENTITIES` `const` | Audit entities Finance may read ("financial" in the role matrix); Management and Admin read everything. |
| `AccessPolicy` `class` | What one signed-in person may do, from their roles and extra rights. |
| `MASKED_ABSENCE` `const` | What a sensitive absence is shown as to people who may not see the reason. |

### packages/domain/src/budget.ts

Budget vs actual and contribution margin.

| | |
|---|---|
| `WARNING_BP` `const` | Health thresholds (D-013 / projects spec): warning from 90 %, over above 100 %. |
| `budgetActual(i: BudgetActualInput)` | Budget vs actual for a project or task: remaining, consumption % (basis points), contribution margin and health. |

### packages/domain/src/calendar-import.ts

Calendar import rules (docs/spec/integrations.md R6) — GRASP Information Expert for turning calendar events into timesheet suggestions: all-day and "free" events are skipped, events spanning midnight are split per day, and hours are rounded to the nearest 0,25 h (never below 0,25 h for a real meeting).

| | |
|---|---|
| `CalendarEventInput` `interface` | A calendar event with local wall-clock times in Europe/Copenhagen ("YYYY-MM-DDTHH:mm[:ss…]"). |
| `EventDay` `interface` | One day's part of an event, as a timesheet suggestion. |
| `roundToQuarter(minutes: number)` | Rounds minutes to centi-hours on the 0,25 h grid; any positive duration becomes at least 0,25 h. |
| `isImportable(ev: CalendarEventInput)` | True when an event should become a suggestion at all (not all-day, not "free", not cancelled). |
| `splitEventByDay(ev: CalendarEventInput)` | Splits an event into one suggestion per day it covers (empty when not importable). |

## packages/domain/src/completion.ts

Fixed-price completion and recognised revenue.

| | |
|---|---|
| `completionBasisPoints(used: CentiHours, budget: CentiHours, overrideBp?: number)` | Fixed-price percentage of completion in basis points: used ÷ budget hours (centi-hours), capped at 100 %. |
| `recognisedRevenue(price: Minor, completionBp: number)` | Revenue recognised on a fixed-price contract: price (øre) × completion (basis points), rounded half away from zero. |

## packages/domain/src/data-import.ts

Data import rules (decision D-024) — GRASP Information Expert for reading customer, project and employee lists from CSV (as saved from Excel): parsing (`;`, `,` or tab, quotes, BOM), Danish/English column names, Danish numbers and dates, and per-row validation.

| | |
|---|---|
| `ImportKind` `type` | What can be imported. |
| `ImportColumn` `interface` | One column: the key we use, accepted header names (lower-case), whether it is required, and an example. |
| `IMPORT_COLUMNS` `const` | Columns per kind. |
| `parseCsv(text: string)` | Splits CSV text into rows of cells: strips a BOM, detects `;`, tab or `,` from the header, honours quotes. |
| `parseNumber(v: string \| undefined)` | "1.234,50" / "1234.5" / "950" → number, or null when empty; NaN when not a number. |
| `parseYesNo(v: string \| undefined)` | "ja/nej", "yes/no", "x", "1/0", "sand/falsk" → boolean; null when empty; undefined when not understood. |
| `parseDate(v: string \| undefined)` | "2026-12-01", "01-12-2026", "1.12.2026" or "01/12/2026" → "2026-12-01"; null when empty or not a real date. |
| `MappedImport` `interface` | Result of reading a file: rows keyed by column key (raw strings), the source line numbers, and problems. |
| `mapImport(kind: ImportKind, table: string[][])` | Maps the header row to column keys (case-insensitive, Danish or English names) and returns keyed rows. |
| `validateRow(kind: ImportKind, v: Record<string, string>)` | Problems found in one row, as error codes (translated in the UI as `imp.err.<code>`). |
| `contractTypeOf(v: string)` | "regning" / "fastpris" / "rammeaftale" (or English) → contract type; null when not understood. |
| `projectStatusOf(v: string)` | "aktiv" / "planlagt" / "afsluttet" / "arkiveret" (or English) → project status; null when not understood. |
| `rolesOf(…)` | "projektleder, leder" → roles (employee is always added by the importer); null when a name is unknown. |
| `importTemplate(kind: ImportKind)` | A template CSV (header + one example row, `;`-separated, with BOM so Excel opens it with æøå intact). |

## packages/domain/src/dates.ts

Local calendar dates as "YYYY-MM-DD" strings in the tenant time zone (Europe/Copenhagen).

| | |
|---|---|
| `LocalDate` `type` | Local calendar dates as "YYYY-MM-DD" strings in the tenant time zone (Europe/Copenhagen). |
| `toUtc(d: LocalDate)` | Parses a LocalDate ("YYYY-MM-DD") into a Date at UTC midnight. |
| `fromUtc(dt: Date)` | Formats a Date as a LocalDate ("YYYY-MM-DD") using its UTC calendar day. |
| `addDays(d: LocalDate, n: number)` | Adds (or subtracts) whole days to a LocalDate. |
| `daysBetween(from: LocalDate, to: LocalDate)` | Number of days from `from` to `to` (negative if `to` is earlier). |
| `isoWeekday(d: LocalDate)` | ISO weekday: Monday = 1 … Sunday = 7. |
| `startOfIsoWeek(d: LocalDate)` | Monday of the ISO week containing the date. |
| `isoWeek(d: LocalDate)` | ISO-8601 week number and week-year. |
| `daysInRange(from: LocalDate, toInclusive: LocalDate)` | All LocalDates from `from` to `toInclusive`, in order. |
| `todayCopenhagen(now: Date = new Date())` | Today's date in Europe/Copenhagen. |

## packages/domain/src/expenses.ts

Expenses and mileage (brief §4.7, docs/spec/expenses.md) — GRASP Information Expert for VAT splitting, mileage allowance and the amount re-invoiced to the customer.

| | |
|---|---|
| `VatSplit` `interface` | Result of splitting a gross (VAT-inclusive) amount. |
| `vatSplit(gross: Minor, vatBp: number, deductibleBp = 10_000)` | Splits a VAT-inclusive amount into net and VAT, and the VAT into deductible and non-deductible parts. |
| `expenseCost` `const` | The company's cost of an expense: net amount + VAT it cannot deduct (øre). |
| `billableAmount(net: Minor, markupBp: number)` | Amount re-invoiced to the customer for a billable expense: net amount plus markup (VAT is added on the invoice). |
| `MileageRate` `interface` | Mileage rates for one year, øre per km; the high rate applies up to `thresholdKm` per person per year. |
| `mileageAmount(metres: number, metresBefore: number, rate: MileageRate)` | Mileage allowance for one trip, taking the yearly threshold into account: kilometres before the person reaches the threshold use the high rate, the rest the low rate. |
| `EXPENSE_STATUSES` `const` | Expense lifecycle — the same as time entries (brief §4.7). |
| `ExpenseStatus` `type` | One of EXPENSE_STATUSES. |

## packages/domain/src/flex.ts

Flex.

| | |
|---|---|
| `weeklyFlex(w: FlexWeekInput)` | Flex for one week in centi-hours: work + absence that counts toward norm − norm hours. |
| `flexBalance(opening: CentiHours, weeks: FlexWeekInput[])` | Flex balance in centi-hours: opening balance + the flex of every given week. |

### packages/domain/src/gdpr.ts

GDPR rules (docs/spec/administration.md R5, decision D-026) — GRASP Information Expert for retention and pseudonymisation.

| | |
|---|---|
| `RETENTION_YEARS` `const` | Years bookkeeping data is retained after the end of its fiscal year. |
| `retentionCutoff(today: LocalDate, years = RETENTION_YEARS)` | First date that is still inside retention: anything dated before it has passed retention. |
| `pseudonymFor(userId: string)` | The pseudonym an erased employee gets: stable for the same person (so reports still group correctly) but not reversible. |
| `ErasureBlocker` `type` | Why an erasure cannot run yet (shown to the Admin, translated as `gdpr.block.<code>`). |
| `erasureBlockers(p: {…})` | Blockers for erasing a person (spec: must be inactive; not PM on active projects; no one reports to them). |

### packages/domain/src/holidays-dk.ts

Built-in Danish public holidays (docs/spec/organisation.md).

| | |
|---|---|
| `easterSunday(year: number)` | Easter Sunday for a year (anonymous Gregorian algorithm). |
| `danishPublicHolidays(year: number)` | Statutory Danish public holidays for a year, sorted by date (Store bededag only up to 2023). |
| `defaultCompanyDays(year: number)` | Default extra company days off (decision D-013); tenants can change these per calendar. |

### packages/domain/src/hours.ts

Hours are handled as integer hundredths of an hour ("centi-hours") to avoid floating-point drift.

| | |
|---|---|
| `CentiHours` `type` | Hours are handled as integer hundredths of an hour ("centi-hours") to avoid floating-point drift. |
| `HoursParseError` `class` | Thrown by `parseHours` when the text is not a valid number of hours. |
| `parseHours(input: string)` | Parses "1,5", "1.5", "1:30", "2" into centi-hours. |
| `isOnStep(h: CentiHours, step: CentiHours = HOURS_STEP)` | True when the centi-hours are a whole multiple of the step (default 25 = 0,25 h). |
| `roundToStep(h: CentiHours, step: CentiHours = HOURS_STEP)` | Rounds to the nearest step; ties round up. |
| `formatHours(h: CentiHours, locale: "da" \| "en" = "da")` | Formats centi-hours with two decimals. |
| `toDecimal` `const` | Centi-hours → decimal hours. |
| `fromDecimal` `const` | Decimal hours → centi-hours (rounded). |

### packages/domain/src/invoicing.ts

Invoice suggestion builder, write-down spreading and totals (brief §4.10, docs/spec/invoicing.md).

| | |
|---|---|
| `isInvoiceable` `const` | Only approved, billable entries not yet on an invoice are invoiceable. |
| `buildTimeLines(…)` | Builds invoice lines from invoiceable time entries, grouped per task, employee, task + employee, or one summary line. |
| `spreadAmount(sources: SourceAllocation[], newAmount: Minor)` | Spreads a changed line amount over its sources proportionally to their registered value, using the largest remainder method so the parts always add up exactly to the new amount. |
| `invoiceTotals(lines: TotalsLine[])` | VAT is calculated per VAT rate on the summed base (not per line) to avoid rounding drift. |
| `dueDate` `const` | Due date from invoice date and payment terms in days. |
| `lineAmount(quantity: number, unitPrice: Minor, discountBp = 0)` | Line amount from quantity (centi-hours or centi-units), unit price and discount (basis points). |
| `AMOUNT_LINE_KINDS` `const` | Invoice line kinds that carry an amount (count in totals/VAT); headings, texts and subtotals do not. |
| `isAmountLine` `const` | True for line kinds that carry an amount. |

### packages/domain/src/kpi.ts

Key figures (brief §4.13, docs/spec/reporting.md) — GRASP Information Expert: one definition of each KPI so the dashboards, reports, exports and the Python analytics service (which mirrors these formulas and is tested against the same examples) all agree.

| | |
|---|---|
| `utilisation` `const` | Utilisation ("udnyttelsesgrad"): billable hours ÷ norm hours. |
| `billablePct` `const` | Billable share ("faktureringsgrad"): billable ÷ all registered work hours (absence excluded). |
| `effectiveRate(invoiced: Minor, hours: CentiHours)` | Effective hourly rate: invoiced amount ÷ invoiced hours, in øre per hour (null without hours). |
| `margin(revenue: Minor, cost: Minor)` | Contribution margin ("dækningsbidrag") and margin % ("DB-grad"). |

### packages/domain/src/money.ts

Money is integer minor units (øre) plus an ISO currency.

| | |
|---|---|
| `assertMinor(n: number)` | Asserts that a number is a safe integer amount in minor units (øre); throws RangeError otherwise. |
| `divRound(numerator: number, denominator: number)` | Integer division with rounding half away from zero. |
| `hoursTimesRate(hours: CentiHours, ratePerHour: Minor)` | hours × hourly rate → amount in minor units. |
| `percentOf(amount: Minor, basisPoints: number)` | amount × percentage in basis points (2500 = 25,00 %). |
| `formatMinor(amount: Minor, locale: "da" \| "en" = "da")` | Formats øre as "1.234,56" (da) or "1,234.56" (en), without currency symbol. |
| `kr` `const` | kr → øre for literals in tests and seed data: kr(925) = 92500. |

## packages/domain/src/norm.ts

Norm (expected) hours.

| | |
|---|---|
| `FULL_TIME_37` const | Default Danish full time: Mon–Thu 7,5 h, Fri 7,0 h = 37 h. |
| `patternHours(date: LocalDate, a: NormAssignment)` | Norm hours (centi-hours) a pattern gives for a date, before holidays; handles 1- and 2-week cycles. |
| `dailyNorm(date: LocalDate, ctx: NormContext)` | Norm hours for one day after the employment period and public holidays. |
| `normForRange(from: LocalDate, days: number, ctx: NormContext)` | Sum of daily norm hours (centi-hours) for `days` consecutive days starting at `from`. |
| `weeklyNorm` const | Norm hours (centi-hours) for the ISO week starting on `monday`, after holidays and employment dates. |

## packages/domain/src/payroll.ts

Payroll export rules (docs/spec/payroll.md, decision D-023) — GRASP Information Expert for turning approved absence, mileage and expenses into payroll lines.

| | |
|---|---|
| `SalaryUnit` type | Unit of a salary code ("lønart"). |
| `absenceDayFraction(hours: CentiHours, normThatDay: CentiHours)` | Absence as days (R2): hours ÷ the employee's norm that day, in hundredths of a day. |
| `PayrollLine` interface | One payroll line: an employee's total on one salary code for one month (or a late delta for an earlier month). |
| `lineKey` const | Key identifying employee × salary code × month. |
| `lateDeltas(current: PayrollLine[], exported: PayrollLine[])` | Late items (R7): for months that are already locked, the difference between what the data says now and what was exported in locked runs. |
| `PAYROLL_CSV_HEADER` const | Columns of the generic CSV (R9). |
| `PayrollCsvRow` interface | One CSV row's input. |
| `payrollCsv(rows: PayrollCsvRow[])` | Generic payroll CSV (R9): UTF-8 with BOM, `;` separator, Danish decimal comma, ISO dates, CRLF line ends. |

## packages/domain/src/periods.ts

Accounting periods (brief §4.11, docs/spec/accounting-periods.md): monthly per legal entity, open → closed.

| | |
|---|---|
| `monthOf` const | "2026-11-08" → "2026-11-01" |
| `isLocked(date: LocalDate, closedMonths: ReadonlySet<LocalDate>)` | True when the date falls in a closed accounting month (`closedMonths` holds first-of-month dates). |
| `lastDayOfMonth(month: LocalDate)` | Last day of the month as a LocalDate. |
| `recognitionDelta(recognisedToDate: Minor, previouslyRecognised: Minor)` | Month-end figures for one fixed-price contract: revenue recognised to date = price × completion; revenue to recognise this month = recognised to date − previously recognised (may be negative after a budget change). |
| `deferredRevenue(invoiced: Minor, usedHours: number, poolHours: number)` | Deferred revenue on a prepaid retainer: invoiced − earned (never negative; an overrun is WIP, not negative deferral). |

### packages/domain/src/planning.ts

Resource planning (brief §4.8, docs/spec/resource-planning.md) — GRASP Information Expert for capacity and utilisation bands.

| | |
|---|---|
| `UtilisationBand` `type` | Heatmap band (brief §4.8): under < 70 %, ok 70–100 % (both inclusive), over > 100 %; none = no capacity. |
| `capacity(normHours: CentiHours, absenceHours: CentiHours)` | Capacity for a week: norm − absence, never below zero. |
| `utilisationBp(booked: CentiHours, cap: CentiHours)` | Booked ÷ capacity in basis points (null when there is no capacity). |
| `utilisationBand(booked: CentiHours, cap: CentiHours)` | Heatmap colour band for a week. |
| `freeHours` `const` | Free hours in a week (capacity − booked, never negative). |

### packages/domain/src/rates.ts

Hourly rate resolution (brief §4.9, docs/spec/rates.md).

| | |
|---|---|
| `matchPriceList(…)` | Most specific matching line of one price list, or undefined. |
| `resolveRate(c: RateContext)` | Resolves the hourly sales rate (øre) for one time entry using the brief's 5-step order and returns which step won. |

### packages/domain/src/roles.ts

Roles are additive (docs/spec/overview.md §3).

| | |
|---|---|
| `ROLES` `const` | Roles are additive (docs/spec/overview.md §3). |
| `hasRole` `const` | True when the user holds at least one of the wanted roles. |

### packages/domain/src/timesheet.ts

Time-entry validation and weekly summary.

| | |
|---|---|
| `isEditableStatus` `const` | Entries the employee may still edit: draft and rejected (rejected returns to draft on edit). |
| `validateEntry(c: EntryCheck)` | Checks one time-entry change against the timesheet rules and returns every error found (empty = valid). |
| `summariseWeek(…)` | Week summary for one employee: per-day norm/work/billable/absence and week totals, flex and billable % (basis points). |
| `dayAnomalies(…)` | Anomaly flags for the approver inbox (docs/spec/approval.md). |

### packages/domain/src/vacation.ts

Danish vacation (samtidighedsferie).

| | |
|---|---|
| `holidayYearOf(d: LocalDate)` | The holiday year (start year) that a date belongs to: 2026-11-08 → 2026 (1 Sep 2026 – 31 Aug 2027). |
| `accruedDays(…)` | Days earned in holiday year `hy` up to and including `asOf`, for an employee employed from `start` (to optional `end`). |
| `vacationBalance(opening: CentiDays, accrued: CentiDays, used: CentiDays)` | Vacation balance = opening + accrued − used (all in centi-days). |

## Database (packages/db) — skema, RLS og arbejdsgange

### packages/db/src/audit.ts

Audit trail writer (brief §4.15: every change to financial data is logged — who, when, before → after, reason).

| | |
|---|---|
| `writeAudit(tx: Tx, e: AuditEntry)` | Writes an audit record inside the caller's transaction, so the change and its audit trail commit together. |

### packages/db/src/client.ts

Database connection and tenant scoping (decision D-012).

| | |
|---|---|
| `createDb(url: string, opts: { max?: number } = {})` | Creates a Drizzle client over a postgres.js pool; `close()` ends the pool. |
| `withTenant(db: Db, tenantId: string, fn: (tx: Tx)` | Runs `fn` in a transaction scoped to one tenant. |

### packages/db/src/invoicing.ts

Invoicing service (brief §4.10, docs/spec/invoicing.md, decisions D-014/D-015).

| | |
|---|---|
| `InvoiceError` `class` | Business error from the invoicing service; `code` is shown to users via i18n, `status` is the HTTP status. |
| `closedMonths(tx: Tx)` | Closed accounting months for the tenant (single legal entity in MVP; per entity in M4). |
| `invoiceableEntries(…)` | Approved, billable, unreserved T&M entries for the given projects up to a date. |
| `createDraftFromSuggestion(tx: Tx, p: DraftParams)` | Creates a draft invoice from everything invoiceable on the projects (one customer). |
| `loadInvoice(tx: Tx, invoiceId: string)` | Loads an invoice header by id or throws InvoiceError("not_found", 404). |
| `recalcInvoice(tx: Tx, invoiceId: string)` | Recomputes subtotal, VAT (per rate) and total from the invoice's amount lines and stores them. |
| `updateLine(tx: Tx, tenantId: string, userId: string, lineId: string, u: LineUpda…)` | Edits a draft invoice line. |
| `addFreeLine(…)` | Adds a manual line (product, text, heading or subtotal) at the end of a draft and recalculates totals. |
| `deleteLine(tx: Tx, lineId: string)` | Deletes a draft line and releases its reserved time entries / milestones so they become invoiceable again. |
| `deleteDraft(tx: Tx, tenantId: string, userId: string, invoiceId: string)` | Deletes an unbooked invoice, releasing all reserved entries and milestones; audited. |
| `bookInvoice(…)` | Books an invoice (D-015): locks the legal entity's counter row, takes the next number, marks sources invoiced, writes audit + outbox — all in the caller's transaction. |
| `createCreditNote(…)` | Creates a draft credit note for a booked invoice: all lines, or the chosen ones, negated. |
| `isDateOpen(tx: Tx, date: LocalDate)` | True when a date may still be changed (its accounting month is open). |

### packages/db/src/migrate.ts

CLI entry point: applies all SQL migrations in packages/db/drizzle as the owner role (`pnpm db:migrate`).

## packages/db/src/periods.ts

Accounting periods and month-end figures (brief §4.11, docs/spec/accounting-periods.md).

| | |
|---|---|
| `PeriodError` `class` | Business error from the accounting-period service; `code` for i18n, `status` for HTTP. |
| `ensurePeriods(…)` | Makes sure a period row exists for every month from `fromMonth` to `toMonth`. |
| `monthEndReport(tx: Tx, month: LocalDate)` | Month-end figures for a month (first day as LocalDate): WIP on T&M (approved, not on a booked invoice), fixed-price recognition (to date, this month, invoiced to date incl. |
| `closeChecklist(tx: Tx, month: LocalDate)` | Pre-close checklist: unapproved timesheets and unbooked invoices dated in the month. |
| `closeMonth(…)` | Closes an accounting month if all earlier months are closed. |
| `reopenMonth(…)` | Reopens the most recently closed month only (audited). |

## packages/db/src/rates.ts

Loads rate-resolution inputs from the database and snapshots sales/cost rates on time entries (brief §4.9, decision D-014).

| | |
|---|---|
| `snapshotRates(tx: Tx, entryIds: string[])` | Resolves and stores the sales rate (billable entries only) and cost rate (øre/h) on the given time entries. |

## packages/db/src/schema.ts

Database schema.

| | |
|---|---|
| `priceGroups` const | Rate role of an employee (e.g. |
| `paymentPlanItems` const | Fixed-price payment plan / retainer fee schedule. |
| `numberSeries` const | Gap-free invoice numbering per legal entity (brief §4.10). |
| `invoiceLineSources` const | Which time entries an invoice line covers, with registered and invoiced value (write-down analytics). |
| `revenueRecognitions` const | Snapshot taken when a month is closed: fixed-price revenue recognised to date and the month's delta. |
| `integrationConnections` const | Configured accounting/payroll connections. |
| `outbox` const | Transactional outbox: events written in the same transaction as the change, delivered by a worker. |
| `expenseTypes` const | Expense categories; `vatDeductibleBp` is the share of the VAT the company may deduct (e.g. |
| `mileageRates` const | Mileage allowance per year: high rate (øre/km) up to the threshold, low rate above. |
| `files` const | Uploaded files (receipts). |
| `expenses` const | An out-of-pocket expense or a mileage claim. |
| `allocations` const | Planned hours: one employee on one project in one ISO week (decision OQ-28 default: weekly granularity). |
| `userCompetencies` const | Competencies as simple tags per employee (e.g. |
| `reminderLog` const | E-mail reminders already sent, so the weekly job never sends the same reminder twice. |
| `graphTokens` const | Delegated Microsoft Graph access token per user (read-only scope Calendars.Read), AES-GCM encrypted with a key derived from SESSION_SECRET. |
| `calendarImports` const | Calendar events (per day) the user has imported into the timesheet or chosen to ignore — shown as handled. |
| `salaryCodes` const | Salary codes ("lønarter") the payroll system knows. |
| `payPeriods` const | Monthly pay periods: open → exported → locked. |
| `payrollRuns` const | One generated export file for a pay period. |
| `payrollLines` const | Lines of a run; `month` is the month the items are dated in (earlier than the run's month for late lines). |
| `userRights` const | Extra rights on top of roles (D-025), e.g. |

## packages/db/src/seed-access.ts

Fictional seed for extra rights (D-025): Anne Lund (AL) and Peter Dahl (PD) may create projects; the other project managers may not.

| | |
|---|---|
| `seedAccess(tx: Tx, tenantId: string)` | Grants the demo rights once. |

## packages/db/src/seed-expenses.ts

M3 part of the fictional seed: expense types, mileage rates and a handful of expenses/mileage claims in the open months (so they show up in approval and in the invoicing basis).

| | |
|---|---|
| `seedExpenses(tx: Tx, tenantId: string, today: LocalDate)` | Seeds expense types, mileage rates and example expenses for the demo tenant (fictional data only). |

### packages/db/src/seed-money.ts

M2 part of the fictional seed: rate groups, task types, price lists, payment plans, booked invoice history for the first months after go-live, and closed accounting periods.

| | |
|---|---|
| `seedMoney(…)` | Seeds the money part of the fictional demo company: price groups, task types, price lists, rate snapshots, payment plans, booked invoice history and closed months. |

### packages/db/src/seed-payroll.ts

M4 part of the fictional seed: salary codes ("lønarter") with illustrative provider codes, the mapping from absence codes, and payroll numbers for every employee except Frederik (FH) — so the export shows the "missing payroll number" warning.

| | |
|---|---|
| `seedPayroll(tx: Tx, tenantId: string)` | Seeds payroll setup for one tenant (no-op when already present). |

### packages/db/src/seed-planning.ts

M3 part of the fictional seed: competencies per employee and allocations for the coming weeks, spread over each employee's customer projects at roughly 55–115 % of a 37-hour week so the heatmap shows all bands.

| | |
|---|---|
| `seedPlanning(tx: Tx, tenantId: string, today: LocalDate)` | Seeds competencies and 8 weeks of allocations from the current ISO week (fictional data only). |

### packages/db/src/seed.ts

Fictional seed data: "ARCnordic Demo ApS" (brief §1 and §9).

## API-services (apps/api/src/services) — arbejdsgange

### apps/api/src/services/approval.ts

Approval service (docs/spec/approval.md, decision D-013: the line manager approves the week) — GRASP Pure Fabrication for the approval workflows.

| | |
|---|---|
| `approvalInbox(tx: Tx, auth: AuthContext, weekStart: LocalDate)` | The approver's inbox for one ISO week: every person the caller approves with status, norm, registered hours, billable %, flex this week, anomaly flags (days far over/under norm) and hours per project. |
| `approvePeriod(tx: Tx, auth: AuthContext, periodId: string)` | Approves a submitted week: submitted entries → approved, rate snapshot taken (5-step resolution), period → approved, event logged and audited. |
| `rejectPeriod(tx: Tx, auth: AuthContext, periodId: string, comment: string)` | Rejects a submitted week with a required comment shown to the employee; entries → rejected (editable again). |
| `reopenApprovedPeriod(tx: Tx, auth: AuthContext, periodId: string, reason: string)` | The approver reopens an approved week (e.g. |

### apps/api/src/services/audit-log.ts

Audit log reader (docs/spec/administration.md R1–R2, decision D-025) — GRASP Pure Fabrication.

| | |
|---|---|
| `AuditFilter` `interface` | Filters for the audit screen; `before` is the cursor (the `at` of the last row already shown). |
| `auditEntities(tx: Tx, auth: AuthContext)` | Entity types the caller may filter on (only those present in the log). |
| `readAuditLog(tx: Tx, auth: AuthContext, f: AuditFilter)` | Newest-first page of audit rows the caller may read, with the actor's name. |

### apps/api/src/services/calendar.ts

Loads an employee's norm context and computes flex and vacation balances from the database.

| | |
|---|---|
| `loadEmployee(tx: Tx, userId: string)` | Loads the user row with employment, rates and opening balances; undefined if not found (RLS-scoped). |
| `normContext(tx: Tx, emp: Employee)` | Builds the domain NormContext for an employee: norm assignments with valid-from, holiday calendar, employment dates. |
| `summaryEntries(…)` | Entries of one employee in a date range, in the shape the domain summary needs. |
| `flexBalance(…)` | Flex balance (docs/spec/absence-flex.md): opening balance at go-live + Σ weekly flex for completed weeks (setting "computed up to last Sunday", D-013) — the current week is shown separately as "this week". |
| `vacationBalance(tx: Tx, emp: Employee, today: LocalDate)` | Vacation (samtidighedsferie, D-013): opening + accrued − used in the current holiday year, in centi-days. |

### apps/api/src/services/customers.ts

Customers (docs/spec/crm.md, light CRM) — GRASP Information Expert for customer records: listing with project counts and creating customers with the next customer number.

| | |
|---|---|
| `listCustomers(tx: Tx)` | All customers with their number of projects, sorted by name. |
| `NewCustomerInput` `interface` | Input for a new customer (CVR 8 digits, EAN 13 digits for public-sector e-invoicing). |
| `createCustomer(tx: Tx, auth: AuthContext, input: NewCustomerInput)` | Creates a customer with the next number in the customer series (1001, 1002 …) and audits it. |

### apps/api/src/services/dashboard.ts

Role-based home dashboard and reminders (brief §4.13, §4.6 reminders) — GRASP Pure Fabrication: gathers, per role, the few things the person should act on ("to-dos") and the key figures they steer by.

| | |
|---|---|
| `Todo` `interface` | One item on the person's to-do list; `count`/`ref` fill the translated text, `href` is where to act. |
| `WeekKpi` `interface` | Week KPI row (decimals for the UI, basis points for percentages). |
| `dashboard(tx: Tx, auth: AuthContext, today: LocalDate)` | The dashboard for the signed-in person: their to-dos (by role) and, for line managers, Finance, Management and Admin, key figures — weekly utilisation/billable % for the people they may see (12 weeks), and invoiced revenue with effective hourly rate for this and last month (Finance/Management/Admin only). |

### apps/api/src/services/data-import.ts

Data import (decision D-024) — GRASP Pure Fabrication: turns a CSV of customers, projects or employees into a plan (create / update per row, with errors), shows it as a dry run, and applies it in one transaction — all rows or none.

| | |
|---|---|
| `PlannedRow` `interface` | One planned row: what will happen and why not, if it cannot. |
| `previewImport(tx: Tx, kind: ImportKind, csv: string)` | Dry run: what the import would do, row by row, without writing anything. |
| `commitImport(tx: Tx, auth: AuthContext, kind: ImportKind, csv: string)` | Applies the import (refused with 422 if any row has errors). |

### apps/api/src/services/errors.ts

Application error used by services and controllers alike (GRASP: Low Coupling — services do not depend on the HTTP/auth layer).

| | |
|---|---|
| `AppError` `class` | Application error used by services and controllers alike (GRASP: Low Coupling — services do not depend on the HTTP/auth layer). |

### apps/api/src/services/expenses.ts

Expenses and mileage (brief §4.7, docs/spec/expenses.md) — GRASP Pure Fabrication for the expense workflows: register (draft), submit, approve/reject, list.

| | |
|---|---|
| `expenseSettings(tx: Tx)` | Expense types and mileage rates for the registration forms. |
| `myExpenses(tx: Tx, auth: AuthContext, month: LocalDate)` | The signed-in employee's expenses in a month (`month` = first day, "YYYY-MM-01"), newest first. |
| `ExpenseInput` `interface` | Input for an expense (`kind: "expense"`, gross amount in kr incl. |
| `saveExpense(tx: Tx, auth: AuthContext, input: ExpenseInput, id?: string)` | Creates or updates (when `id` is given and still draft/rejected) an expense for the signed-in employee and computes its amounts: VAT split by expense type for expenses; allowance from the yearly rate table for mileage. |
| `deleteExpense(tx: Tx, auth: AuthContext, id: string)` | Deletes one of the employee's own draft/rejected expenses. |
| `submitExpenses(tx: Tx, auth: AuthContext, ids?: string[])` | Submits the employee's draft/rejected expenses (all of them, or the given ids) for approval. |
| `expenseApprovalQueue(tx: Tx, auth: AuthContext)` | Submitted expenses waiting for the caller: their reports' (line manager) or everyone's (Finance, D-025). |
| `approveExpense(tx: Tx, auth: AuthContext, id: string)` | Approves a submitted expense (audited). |
| `rejectExpense(tx: Tx, auth: AuthContext, id: string, comment: string)` | Rejects a submitted expense with a required comment shown to the employee (audited). |
| `canSeeFile(tx: Tx, auth: AuthContext, fileId: string)` | True if the caller may open the receipt file: its owner, the approver of the expense, or Finance/Admin. |

### apps/api/src/services/files.ts

File storage for receipts (brief §4.7) — GRASP Pure Fabrication + Protected Variations: callers only see `storeFile` / `readFile`; today bytes are kept on the local disk (`data/uploads/<tenant>/<file id>`, D-017 runs everything locally), later this module can switch to Azure Blob/SharePoint without touching callers.

| | |
|---|---|
| `MAX_FILE_BYTES` `const` | Maximum receipt size in bytes (10 MB). |
| `sniffContentType(bytes: Buffer)` | Detects the real content type from the first bytes; null when the type is not allowed. |
| `storeFile(tx: Tx, auth: AuthContext, filename: string, bytes: Buffer)` | Validates and stores an uploaded file for the signed-in user; returns the file row (id used by expenses). |
| `readStoredFile(tx: Tx, auth: AuthContext, fileId: string)` | Reads a stored file if the caller may see it: its owner, or anyone who may see an expense that references it (checked by the caller via `canSeeFile`). |

### apps/api/src/services/gdpr.ts

GDPR workflows (docs/spec/administration.md, decision D-026) — GRASP Pure Fabrication: data-subject export, erasure (pseudonymisation) of former employees, and the nightly retention sweep.

| | |
|---|---|
| `exportPersonalData(tx: Tx, auth: AuthContext, userId: string)` | Everything ARCnordic Tid holds about one person, as one JSON document (spec: data-subject export; AC5 — nothing about others). |
| `erasurePreview(tx: Tx, userId: string)` | What an erasure would do and whether it is blocked (shown to the Admin before confirming). |
| `erasePerson(tx: Tx, auth: AuthContext, userId: string, reason: string)` | Erases a former employee (spec R5, AC4): refuses while blocked, then pseudonymises the profile, clears comments on kept entries, and deletes non-bookkeeping personal data. |
| `retentionSweep(tx: Tx, today: LocalDate)` | Nightly retention sweep for one tenant: deletes short-lived personal data (expired sessions and Graph tokens, reminder and calendar-import logs older than 13 months) and reports bookkeeping records past the 5-year retention, which Finance purges deliberately (D-026) rather than a job deleting accounting data on its own. |
| `pastRetention(tx: Tx, today: LocalDate)` | Bookkeeping records dated before the retention cutoff, and former employees not yet erased. |
| `runRetentionSweep(db: Db, today: LocalDate = todayCopenhagen()` | Runs the retention sweep for every tenant (background job; each tenant in its own RLS-scoped transaction). |

### apps/api/src/services/invoice-document.ts

Invoice document (HTML → PDF, decision D-016).

| | |
|---|---|
| `invoiceHtml(tx: Tx, invoiceId: string)` | Renders the invoice/credit note as a self-contained Danish HTML document (KLADDE watermark until booked, optional time-specification appendix). |
| `htmlToPdf(html: string)` | Renders HTML to an A4 PDF with headless Chromium (one shared browser per process). |

## apps/api/src/services/invoices.ts

Invoice access and read models for the API (brief §4.10, docs/spec/invoicing.md) — GRASP Pure Fabrication.

| | |
|---|---|
| `assertCanInvoice(tx: Tx, auth: AuthContext, projectIds: string[])` | Throws 403 unless the caller may prepare invoices for all given projects (Finance, or PM of each project). |
| `assertCanSeeInvoice(tx: Tx, auth: AuthContext, invoiceId: string)` | Loads an invoice the caller may see, or throws 404 (not 403, so existence is not revealed). |
| `assertCanSeeLine(tx: Tx, auth: AuthContext, lineId: string)` | Loads the invoice that owns a line and checks the caller may see it; returns the invoice id. |
| `invoicingBasis(tx: Tx, auth: AuthContext, upTo: LocalDate)` | Invoicing basis ("faktureringsgrundlag"): per customer project, everything invoiceable up to `upTo` — approved, billable, unreserved T&M hours (value at the approval rate snapshot, øre), due or ready payment-plan items, and approved billable expenses (net + markup). |
| `activeCustomerProjects(tx: Tx)` | Active customer projects, for one-click bulk drafting. |
| `listInvoices(…)` | Invoice list filtered by status ("open" = not booked), newest number first; PMs see their projects only. |
| `invoiceDetail(tx: Tx, auth: AuthContext, invoiceId: string)` | Full invoice for the editor: header, customer, lines with how many time entries each covers, VAT totals, credit notes. |
| `InvoiceHeaderUpdate` `interface` | Editable invoice header fields. |
| `updateInvoiceHeader(…)` | Updates header fields of an unbooked invoice (audited). |
| `markReady(tx: Tx, auth: AuthContext, invoiceId: string)` | Marks a draft "ready for booking" (lets a PM hand over to Finance; decision D-013 / observed separation of roles). |

## apps/api/src/services/outbox.ts

Outbox delivery (transactional outbox, docs/spec/integrations.md).

| | |
|---|---|
| `processOutbox(…)` | Claims up to 20 pending outbox events across tenants and delivers them (e.g. |

## apps/api/src/services/outlook.ts

Outlook calendar import (docs/spec/integrations.md R6) — GRASP Pure Fabrication + Indirection: shows a week's meetings next to the timesheet and turns the ones the user picks into draft hours on a chosen task.

| | |
|---|---|
| `CalendarProvider` `interface` | Where the calendar comes from. |
| `CalendarStatus` `type` | disabled = not configured · not_connected = user must sign in with Microsoft · connected = events listed |
| `encryptToken(plain: string, secret: string)` | Encrypts a token as "iv.tag.ciphertext" (base64url, AES-256-GCM). |
| `decryptToken(value: string, secret: string)` | Reverses encryptToken; throws if the value was tampered with or the secret changed. |
| `storeGraphToken(…)` | Stores (or replaces) the user's delegated Graph token after Entra sign-in. |
| `graphCalendarProvider(enabled: boolean, secret: string)` | The production provider: Microsoft Graph with the stored token (disabled unless configured). |
| `CalendarItem` `interface` | One meeting-day as shown in the UI (hours as decimals), with what the user already did with it. |
| `weekCalendar(…)` | The week's meetings for the signed-in user, split per day and rounded to 0,25 h, marked as new/imported/ignored. |
| `ImportItem` `interface` | A meeting-day the user wants on the timesheet. |
| `importCalendarItems(tx: Tx, auth: AuthContext, items: ImportItem[])` | Adds each picked meeting as draft hours on the chosen task (added to what the cell already holds; the meeting subject becomes the comment when the cell has none) and marks it imported. |
| `ignoreCalendarItem(tx: Tx, auth: AuthContext, eventId: string, date: LocalDate)` | Marks a meeting-day as "not time to register" so it no longer shows as new. |

## apps/api/src/services/payroll.ts

Payroll export (docs/spec/payroll.md, decision D-023) — GRASP Pure Fabrication: gathers approved absence, mileage and personal expenses per employee × salary code × month, previews them with warnings, writes the generic CSV and locks months.

| | |
|---|---|
| `payPeriods(tx: Tx, auth: AuthContext)` | Creates monthly pay periods from the go-live month up to the current month (idempotent) and lists them. |
| `PreviewLine` `interface` | One preview line for the UI (quantities as decimals, amounts in øre). |
| `payrollPreview(tx: Tx, auth: AuthContext, month: LocalDate)` | Preview of a month's export: one row per employee with their lines and warnings — missing payroll number (row not exported, AC2) and items still awaiting approval (they become late lines once approved). |
| `exportPayroll(tx: Tx, auth: AuthContext, month: LocalDate)` | Generates (or regenerates) the export for a month: employees without a payroll number are left out, the file replaces any earlier unlocked run (AC4), and the period becomes "exported". |
| `lockPayPeriod(tx: Tx, auth: AuthContext, month: LocalDate)` | Locks an exported month: its lines are frozen; later changes to it become late lines (R8). |
| `payrollFile(tx: Tx, month: LocalDate)` | The CSV file of a month's run (404 when not exported yet). |

### apps/api/src/services/people.ts

Who may see or approve whom (docs/spec/overview.md §3, decision D-013).

| | |
|---|---|
| `reportIds(tx: Tx, managerId: string)` | Direct and indirect reports of a manager. |
| `visibleUserIds(tx: Tx, auth: AuthContext)` | Users whose time the caller may read: self, reports (line manager), everyone (admin/finance/management). |
| `relationTo(…)` | How the caller relates to an employee, for privacy rules (AccessPolicy.seesAbsenceReason): the employee themself, their line manager (direct or indirect), or anyone else. |
| `canViewUser(tx: Tx, auth: AuthContext, userId: string)` | True if the caller may read the user's time data (self, a report of a line manager, or Admin/Finance/Management). |
| `canApproveUser(tx: Tx, auth: AuthContext, userId: string)` | Approver rule (D-013, D-025): the direct line manager approves. |
| `approveeIds(tx: Tx, auth: AuthContext)` | Users the caller approves (for the inbox). |

### apps/api/src/services/planning.ts

Resource planner (brief §4.8, docs/spec/resource-planning.md) — GRASP Pure Fabrication: builds the capacity heatmap (capacity vs booked per employee per ISO week) and changes allocations.

| | |
|---|---|
| `PlannerWeek` `interface` | One week cell in the heatmap (hours as decimals for the UI). |
| `plannerView(…)` | The heatmap for `weeks` ISO weeks from `fromMonday`: per employee their weeks (capacity, booked, free, band), allocation rows per project, competencies; optionally filtered by competency (case-insensitive substring). |
| `setAllocation(…)` | Sets planned hours (decimal) for an employee on a project in one ISO week; 0 removes the allocation. |
| `setCompetencies(tx: Tx, auth: AuthContext, userId: string, names: string[])` | Replaces an employee's competency tags (Admin or their line manager). |
| `allCompetencies(tx: Tx)` | Distinct competency names in the company (for the filter suggestions). |

### apps/api/src/services/project-approval.ts

Project manager approval step (docs/spec/approval.md, mode pm_then_line_manager, decision D-022) — GRASP Pure Fabrication.

| | |
|---|---|
| `ApprovalMode` `type` | How weeks are approved in this tenant. |
| `approvalMode(tx: Tx, tenantId: string)` | The tenant's approval mode (defaults to line manager only). |
| `pendingPmSteps(tx: Tx, userId: string, weekStart: LocalDate)` | Number of open PM steps (distinct projects) in one employee's week — shown to the line manager. |
| `pmInbox(tx: Tx, auth: AuthContext)` | The project manager's inbox: per employee × week × project, the submitted hours waiting for them, with the individual entries (date, task, hours, comment). |
| `PmDecision` `interface` | One PM decision: an employee's week on one project. |
| `pmDecide(tx: Tx, auth: AuthContext, d: PmDecision)` | Records the project manager's decision. |

### apps/api/src/services/project-setup.ts

Project setup workflows (docs/spec/projects.md) — GRASP Creator + Pure Fabrication: creating projects (with their first contract and the PM as member), adding tasks to the WBS and editing tasks.

| | |
|---|---|
| `WBS_MAX_LEVEL` `const` | Maximum WBS depth (brief §4.3). |
| `NewProjectInput` `interface` | Input for a new project; money in kroner (converted to øre here). |
| `createProject(tx: Tx, auth: AuthContext, b: NewProjectInput)` | Creates an active project with one contract of the chosen type and the PM as member (audited). |
| `NewTaskInput` `interface` | Input for a new task; budget amount in kroner. |
| `addTask(tx: Tx, auth: AuthContext, projectId: string, b: NewTaskInput)` | Adds a task to the project's WBS (max 5 levels). |
| `TaskUpdate` `interface` | Editable task fields; budget amount in kroner. |
| `updateTask(tx: Tx, auth: AuthContext, taskId: string, patch: TaskUpdate)` | Updates a task (name, budget, open/closed, billable) — only the project's PM, Finance or Admin (audited). |

### apps/api/src/services/projects.ts

Project scope and budget-vs-actual aggregation (docs/spec/projects.md, reporting.md).

| | |
|---|---|
| `visibleProjectIds(tx: Tx, auth: AuthContext)` | Projects the caller may see: all (admin/finance/management), otherwise managed + member projects. |
| `canEditProject(tx: Tx, auth: AuthContext, projectId: string)` | True if the caller may change the project: Admin/Finance, or its project manager. |
| `taskActuals(tx: Tx, projectIds: string[])` | Actuals per task. |
| `projectSummaries(tx: Tx, projectIds: string[] \| "all")` | Project list rows with budget vs actual (hours, value, cost, margin, health) for the given ids or all projects. |
| `projectDetail(tx: Tx, projectId: string)` | Task tree with budget vs actual rolled up from leaves, plus contract completion for fixed price. |

### apps/api/src/services/reminders.ts

Weekly e-mail reminders (brief §4.6 "reminders") — GRASP Pure Fabrication + Indirection: a job that, on Monday mornings (Europe/Copenhagen), reminds employees who have not submitted last week and approvers who have timesheets waiting.

| | |
|---|---|
| `Mailer` `interface` | Minimal mail-sending interface (nodemailer in production, a fake in tests). |
| `mailerFromEnv(env: Record<string, string \| undefined>)` | Builds an SMTP mailer from env (SMTP_HOST/SMTP_PORT/MAIL_FROM), or null when not configured — the default. |
| `isReminderWindow(now: Date = new Date())` | True from Monday 09:00 Copenhagen time until the end of Monday — the window the weekly job may run in. |
| `sendTenantReminders(…)` | Sends this week's reminders for one tenant; returns how many e-mails were sent. |
| `runWeeklyReminders(…)` | Runs the weekly reminders for every tenant if we are in the Monday window and a mailer is configured. |

## apps/api/src/services/reports.ts

Reports and balances (docs/spec/reporting.md) — GRASP Pure Fabrication: report queries and CSV export.

| | |
|---|---|
| `toCsv(header: string[], rows: (string \| number \| null)` | Builds a CSV document for Danish Excel. |
| `myBalances(tx: Tx, auth: AuthContext)` | The signed-in employee's flex balance (hours, completed weeks) and vacation balance (days) for the holiday year. |
| `budgetActualRows(tx: Tx, auth: AuthContext)` | Budget vs actual for every project the caller may see (same figures as the project list). |
| `budgetActualCsv(rows: Awaited<ReturnType<typeof budgetActualRows>>)` | Budget vs actual as a Danish CSV document. |
| `TimeEntryFilter` `interface` | Filters for the time-entry report. |
| `timeEntryRows(tx: Tx, auth: AuthContext, f: TimeEntryFilter)` | Time entries in a date range for the people the caller may see (self; reports for a line manager; everyone for Finance/Admin/Management). |
| `timeEntryCsv(rows: Awaited<ReturnType<typeof timeEntryRows>>)` | Time entries as a Danish CSV document. |

### apps/api/src/services/timesheet.ts

Timesheet service (docs/spec/time-tracking.md) — GRASP Pure Fabrication: the database workflows behind the weekly timesheet.

| | |
|---|---|
| `LOCKED_PERIOD` `const` | Timesheet period statuses in which the employee can no longer edit entries. |
| `tenantGoLive(tx: Tx, tenantId: string)` | The tenant's go-live date: balances are opening balance at this date + registrations after it. |
| `getPeriod(tx: Tx, userId: string, weekStart: LocalDate)` | The employee's timesheet period for the ISO week starting `weekStart` (Monday), or undefined. |
| `ensurePeriod(tx: Tx, auth: AuthContext, userId: string, weekStart: LocalDate)` | Returns the period for the week, creating an "open" one if none exists (GRASP Creator: the week owns its period). |
| `taskRows(tx: Tx, taskIds: string[])` | Task + project information for the given task ids (used for row labels and entry validation). |
| `weekView(tx: Tx, auth: AuthContext, userId: string, weekStart: LocalDate)` | Builds everything the week screen shows for one employee and week: rows (task/absence) with cells per day, norm per day, holidays, closed days, week summary (hours, billable %, flex this week), flex balance and the event log. |
| `CellInput` `interface` | One cell of the timesheet: a task OR an absence code on a date, with decimal hours (0 clears the cell). |
| `saveCell(tx: Tx, auth: AuthContext, b: CellInput)` | Saves one timesheet cell for the signed-in employee: checks membership and leaf task, runs the shared domain validation (24 h/day, 0,25 step, future work, closed project/task, required comment, locked period), then creates, updates or clears the entry. |
| `submitWeek(tx: Tx, auth: AuthContext, weekStart: LocalDate, comment: string \| nu…)` | Submits the employee's week for approval: draft/rejected entries → submitted, period → submitted, event logged. |
| `withdrawWeek(tx: Tx, auth: AuthContext, weekStart: LocalDate)` | Lets the employee take back a submitted week before anyone has approved it (audited). |
| `assignableTasks(tx: Tx, auth: AuthContext, search?: string)` | Tasks the employee can register on: open leaf tasks on active projects they are a member of; optional text search. |
| `addFavourite(tx: Tx, auth: AuthContext, taskId: string)` | Marks a task as one of the employee's favourites (idempotent). |
| `removeFavourite(tx: Tx, auth: AuthContext, taskId: string)` | Removes a task from the employee's favourites. |
| `previousWeekRows(tx: Tx, auth: AuthContext, weekStart: LocalDate)` | The still-usable task rows the employee had the week before `weekStart` ("copy last week's rows" — rows only, no hours). |

## API-ruter og login (apps/api/src/routes, auth)

### apps/api/src/app.ts

Fastify application factory — wires security plugins (helmet, CORS, cookies, rate limit), OpenAPI docs, health checks, the auth plugin and all /api/v1 route modules, plus the global error handler that turns AppError, invoice/period errors and Postgres check violations into `{ error: code }` responses.

| | |
|---|---|
| `AppDeps` `interface` | Replaceable collaborators (tests pass fakes); defaults are the real, config-driven implementations. |
| `buildApp(config: Config, db: Db, deps: AppDeps = {})` | Builds the configured Fastify app (no listening) — injected config and database make it testable with `app.inject`. |

### apps/api/src/auth/plugin.ts

Authentication + CSRF.

| | |
|---|---|
| `AuthContext` interface | The signed-in person for one request. |
| `policyOf` const | The access policy for a request's person (the role matrix lives in packages/domain/src/access.ts). |
| `authPlugin` const | Fastify plugin: resolves the session cookie to `request.auth`, adds `request.withTenant()`, and enforces login and the CSRF header on every route not marked `config: { public: true }`. |
| `requireRole(req: FastifyRequest, ...roles: Role[])` | Returns the auth context if the caller has at least one of the roles, otherwise throws 401/403. |
| `requireCapability(req: FastifyRequest, capability: Capability)` | Throws 401/403 unless the caller's access policy allows the capability; returns the auth context. |

### apps/api/src/auth/routes.ts

Authentication routes — GRASP Controller: dev login (email + password, never in production), logout, and Microsoft Entra ID sign-in via OIDC with PKCE (enabled only when configured).

| | |
|---|---|
| `authRoutes` const | Registers /auth/providers, /auth/dev-login (dev only), /auth/logout and the Entra ID login/callback routes. |

### apps/api/src/auth/session.ts

Session store — opaque random tokens in an httpOnly cookie; only their SHA-256 hash is stored in the database, together with a per-session CSRF token.

| | |
|---|---|
| `sha256` const | Hex SHA-256 of a string (used so raw session tokens are never stored). |
| `createSession(…)` | Creates a 12-hour session for the user in their tenant; returns the raw token (for the cookie) and the CSRF token. |
| `readSession(db: Db, raw: string)` | Looks up a valid, unexpired session for an active user by raw token; null if none. |
| `deleteSession(db: Db, raw: string, tenantId: string)` | Deletes the session (logout). |
| `safeEqual(a: string, b: string)` | Constant-time string comparison (CSRF token check) to avoid timing leaks. |

### apps/api/src/config.ts

Environment configuration, validated with Zod at start-up (fails fast on missing/invalid values).

| | |
|---|---|
| `loadConfig(overrides: Record<string, string \| undefined> = {})` | Loads .env from the repo root, applies overrides (used by tests) and returns the validated config. |

### apps/api/src/routes/admin.ts

Administration (brief §4.15): norm-time patterns, holiday calendars, absence codes, employee settings.

| | |
|---|---|
| `adminRoutes` const | Registers the administration endpoints (settings, norm patterns, holidays, absence codes, employee settings). |

### apps/api/src/routes/approval.ts

Approval routes (docs/spec/approval.md) — GRASP Controller: validate input and delegate to `services/approval.ts`, which also checks that the caller is the approver.

| | |
|---|---|
| `approvalRoutes` const | Registers the approval endpoints under /api/v1. |

### apps/api/src/routes/calendar.ts

Outlook calendar import routes (docs/spec/integrations.md R6) — GRASP Controller: validate input and delegate to `services/outlook.ts`.

| | | |
|---|---|---|
| `calendarRoutes` | `const` | Registers the calendar import endpoints under /api/v1. |

### apps/api/src/routes/data-import.ts

Data import routes (D-024) — GRASP Controller: checks the role and delegates to `services/data-import.ts`.

| | | |
|---|---|---|
| `dataImportRoutes` | `const` | Registers the import endpoints under /api/v1. |

### apps/api/src/routes/expenses.ts

Expense and receipt routes (brief §4.7, docs/spec/expenses.md) — GRASP Controller: validate input, delegate to `services/expenses.ts` and `services/files.ts`.

| | | |
|---|---|---|
| `expenseRoutes` | `const` | Registers the expense and file endpoints under /api/v1. |

### apps/api/src/routes/finance.ts

Accounting periods, month-end report, price lists and payment plans (brief §4.9, §4.11).

| | | |
|---|---|---|
| `financeRoutes` | `const` | Registers accounting-period, month-end report, price-list and payment-plan endpoints. |

### apps/api/src/routes/governance.ts

Audit log and GDPR routes (docs/spec/administration.md, D-025/D-026) — GRASP Controller: check the capability, validate input, delegate to `services/audit-log.ts` and `services/gdpr.ts`.

| | | |
|---|---|---|
| `governanceRoutes` | `const` | Registers the audit and GDPR endpoints under /api/v1. |

### apps/api/src/routes/invoicing.ts

Invoicing routes (brief §4.10, docs/spec/invoicing.md) — GRASP Controller.

| | | |
|---|---|---|
| `invoicingRoutes` | `const` | Registers the invoicing endpoints under /api/v1. |

### apps/api/src/routes/payroll.ts

Payroll export routes (docs/spec/payroll.md, D-023) — GRASP Controller: Finance only (AC7); validate input and delegate to `services/payroll.ts`.

| | | |
|---|---|---|
| `payrollRoutes` | `const` | Registers the payroll endpoints under /api/v1. |

### apps/api/src/routes/planning.ts

Resource planner routes (brief §4.8) — GRASP Controller: validate input, delegate to `services/planning.ts`.

| | | |
|---|---|---|
| `planningRoutes` | `const` | Registers the resource planner endpoints under /api/v1. |

### apps/api/src/routes/projects.ts

Project, task and customer routes (docs/spec/projects.md, crm.md) — GRASP Controller: validate input, check the role, delegate to services (`projects.ts` read models, `project-setup.ts` changes, `customers.ts`).

| | | |
|---|---|---|
| `projectRoutes` | `const` | Registers the project, task and customer endpoints under /api/v1. |

### apps/api/src/routes/reports.ts

Report routes (docs/spec/reporting.md) — GRASP Controller: validate filters, delegate to `services/reports.ts`, and choose JSON or CSV download.

| | | |
|---|---|---|
| `reportRoutes` | `const` | Registers the report endpoints under /api/v1. |

### apps/api/src/routes/timesheet.ts

Timesheet routes (docs/spec/time-tracking.md) — GRASP Controller: validate input, check access, delegate to `services/timesheet.ts`.

| | |
|---|---|
| `timesheetRoutes` `const` | Registers the timesheet endpoints under /api/v1. |

### apps/api/src/routes/users.ts

People endpoints.

| | |
|---|---|
| `userRoutes` `const` | Registers /me, /users (scoped by role) and /users/:id/roles (admin, audited). |

### apps/api/src/server.ts

Process entry point for the API (`pnpm dev` / `pnpm start`): builds the app, listens on API_PORT, runs the outbox delivery loop every 30 s, weekly reminders, a daily retention sweep, and shuts down cleanly on SIGINT/SIGTERM.

## Integrationer (packages/integrations)

### packages/integrations/src/adapter.ts

Accounting integration adapter (brief §4.14, docs/spec/integrations.md).

| | |
|---|---|
| `CustomerDoc` `interface` | Accounting integration adapter (brief §4.14, docs/spec/integrations.md). |
| `IntegrationError` `class` | Error from an external system. |

### packages/integrations/src/calendar.ts

Calendar adapter (docs/spec/integrations.md R6) — GRASP Polymorphism + Protected Variations: the app only knows `CalendarSource`; Microsoft Graph is one implementation.

| | |
|---|---|
| `CalendarEvent` `interface` | A calendar event with local wall-clock times in Europe/Copenhagen ("YYYY-MM-DDTHH:mm:ss…"). |
| `CalendarSource` `interface` | Anything that can list a person's events between two dates (from inclusive, to exclusive, "YYYY-MM-DD"). |
| `fromGraphEvent(e: GraphEvent)` | Maps a Graph event to our neutral shape (pure; tested without network). |
| `GraphCalendarSource` `class` | Reads the signed-in user's calendar through Microsoft Graph with a delegated access token. |

### packages/integrations/src/economic.ts

e-conomic adapter (REST API, https://restapi.e-conomic.com).

| | |
|---|---|
| `toEconomicCustomer` `const` | Maps our customer document to the e-conomic customer payload (pure function, tested without network). |
| `toEconomicDraft` `const` | Amounts in e-conomic are decimal kroner; quantities may be negative on credit notes. |
| `EconomicAdapter` `class` | AccountingAdapter for e-conomic's REST API. |
| `economicFromEnv(env: Record<string, string \| undefined>)` | Builds the adapter from environment variables, or null when not configured (the default). |

## Web: komponenter og hjælpere (apps/web/src)

### apps/web/src/components/app-shell.tsx

Application frame: top bar (wordmark, language, user, sign out) and the role-filtered main navigation.

| | |
|---|---|
| `AppShell({ children }: { children: ReactNode })` | Wraps every signed-in page with the top bar, navigation and a skip-to-content link. |

### apps/web/src/components/dashboard.tsx

Home-page dashboard pieces (pure view components): the role-based to-do list and the key-figure panel, both fed by GET /me/dashboard (apps/api/src/services/dashboard.ts).

| | |
|---|---|
| `Todo` `interface` | One to-do as returned by the API. |
| `WeekKpi` `interface` | One complete ISO week of key figures (hours as decimals, percentages in basis points). |
| `MonthRevenue` `interface` | Invoiced revenue for one month (øre), with the effective hourly rate (øre per hour). |
| `DashboardData` `interface` | Shape of GET /me/dashboard. |
| `TodoList({ todos }: { todos: Todo[] })` | The person's to-do list; each line links to where it is solved. |
| `KpiPanel({ kpis }: { kpis: NonNullable<DashboardData["kpis"]> })` | Key figures for the team or company: utilisation, billable share, a 12-week trend and (Finance) revenue. |

### apps/web/src/components/expense-approvals.tsx

Expense approval list for line managers and Finance (brief §4.7, decision D-019): each submitted expense with receipt link, approve, or reject with a required comment.

| | |
|---|---|
| `ExpenseApprovals()` | Lists submitted expenses waiting for the signed-in approver. |

### apps/web/src/components/gdpr-section.tsx

GDPR actions for one employee (docs/spec/administration.md, D-026), shown to Admin in the employee panel: download everything about the person (data-subject export), and erase a former employee — with a preview of what is kept (bookkeeping, pseudonymised) and removed, blockers explained, a required reason and a confirmation.

| | |
|---|---|
| `GdprSection({ userId }: { userId: string })` | Export + erasure controls for the employee with `userId`. |

### apps/web/src/components/language-switch.tsx

DA/EN language toggle.

| | |
|---|---|
| `LanguageSwitch()` | Two pressed-state buttons (Dansk / English) that switch the UI language immediately. |

### apps/web/src/components/new-project.tsx

"Nyt projekt" form on the projects page (docs/spec/projects.md).

| | |
|---|---|
| `NewProject({ onCancel }: { onCancel: ()` | Short form: the essentials only. |

### apps/web/src/components/payment-plan.tsx

Payment plan card on the project page: fixed-price milestones / retainer fees (brief §4.3, §4.10).

| | |
|---|---|
| `PaymentPlan({…})` | Fixed-price payment plan / retainer fee schedule on the project page (brief §4.3). |

### apps/web/src/components/progress.tsx

Budget consumption bar: neutral up to 90 %, amber 90–100 %, deep orange over budget (never brand red).

| | |
|---|---|
| `Progress({ bp, label }: { bp: number \| null; label: string })` | Budget consumption bar: neutral up to 90 %, amber 90–100 %, deep orange over budget (never brand red). |
| `HealthDot({ health, label }: { health: string; label: string })` | Small coloured dot + label for budget health (ok, warning, over, no budget); colour is never the only signal. |

### apps/web/src/components/project-approvals.tsx

Project manager approval list (docs/spec/approval.md, mode pm_then_line_manager, decision D-022): each employee's submitted hours on the caller's projects, grouped per week and project, with the entries behind them.

| | |
|---|---|
| `ProjectApprovals()` | Lists the project manager's open approval steps. |

### apps/web/src/components/session.tsx

Session context for the signed-in user.

| | |
|---|---|
| `SessionGate({ children, fallback }: { children: ReactNode; fallback: ReactNode })` | Loads the signed-in user; redirects to /login when there is no session. |
| `useMe()` | Returns the signed-in user (with roles and tenant). |

### apps/web/src/components/timesheet/calendar-panel.tsx

"From my calendar" panel on the timesheet (docs/spec/integrations.md R6): lists the week's Outlook meetings (read only), lets the employee tick some and register them as draft hours on one task, or ignore them.

| | |
|---|---|
| `CalendarPanel({ weekStart, onClose }: { weekStart: string; onClose: ()` | The calendar panel for one week; `onClose` hides it. |

### apps/web/src/components/timesheet/day-view.tsx

Day view (docs/spec/time-tracking.md): one day at a time, hours and comment side by side — handy when writing comments or registering on a phone-sized screen.

| | |
|---|---|
| `DayView({…})` | Day view of the timesheet: day picker for the week plus one row per task with hours and comment side by side. |

### apps/web/src/components/timesheet/task-picker.tsx

Accessible combobox for adding a task row (WAI-ARIA combobox pattern): type to search, ↑↓ to move, Enter to add, Esc to close.

| | |
|---|---|
| `TaskPicker({ onPick, exclude }: { onPick: (t: TaskRef)` | Accessible combobox for adding a task row (WAI-ARIA pattern): type to search, ↑↓ to move, Enter to add, Esc to close. |

### apps/web/src/components/timesheet/types.ts

Shared types for the timesheet screens, mirroring the GET /api/v1/timesheet/week/:weekStart response.

| | |
|---|---|
| `TaskRef` `interface` | Shared types for the timesheet screens, mirroring the GET /api/v1/timesheet/week/:weekStart response. |
| `taskRowKey` `const` | Stable row key for a task row in the timesheet grid. |
| `absenceRowKey` `const` | Stable row key for an absence-code row in the timesheet grid. |

### apps/web/src/components/timesheet/week-grid.tsx

Weekly timesheet grid (docs/spec/time-tracking.md).

| | |
|---|---|
| `rowLabel` `const` | Human-readable label for a timesheet row (used for accessible cell names and panels). |
| `WeekGrid({ week, rows, selected, onSelect, onCommentNeeded }: Props)` | Keyboard-first weekly timesheet grid. |

### apps/web/src/lib/api.ts

Thin client for the ARCnordic Tid API.

| | |
|---|---|
| `ANALYTICS_URL` `const` | Base URL of the Python analytics service (KPI Excel export, Power BI datasets). |
| `setCsrfToken` `const` | Remembers the session's CSRF token (from login or /me); it is sent on every mutating request. |
| `ApiError` `class` | Error thrown by `api` for non-2xx responses; `code` is the API's error code (translate with `err.<code>`). |
| `api(path: string, init: RequestInit = {})` | Calls the ARCnordic Tid API under /api/v1 with the session cookie, JSON body and CSRF header; returns parsed JSON. |
| `can` `const` | True when the signed-in person may do the action — the single check the web uses (D-025). |
| `UploadedFile` `interface` | Metadata of an uploaded file (receipt) as returned by POST /files. |
| `uploadFile(file: File)` | Uploads one file (e.g. |

### apps/web/src/lib/format.ts

Display formatting for the web app (Danish first): hours, money, percentages, dates and week labels.

| | |
|---|---|
| `fmtHours(h: number \| null \| undefined, l: Locale, blankZero = false)` | 7.5 → "7,50" (da) / "7.50" (en); 0 → "" when blankZero. |
| `fmtFlex(h: number, l: Locale)` | Signed hours for flex: +0,50 / −1,25 |
| `fmtKr(minor: number \| null \| undefined, l: Locale, decimals = 0)` | øre → "117.600 kr" (whole kroner for dashboards) |
| `fmtPct(bp: number \| null \| undefined, l: Locale)` | basis points → "49,0 %" |
| `fmtDay(date: string, l: Locale, opts: Intl.DateTimeFormatOptions)` | Formats a LocalDate with Intl options in the UI locale (UTC-based, so the day never shifts). |
| `weekLabel(monday: string, l: Locale)` | Label for a week, e.g. |

## Python-analysetjeneste (apps/analytics)

### apps/analytics/arc_analytics/db.py

Database access for the analytics service — GRASP Pure Fabrication.

| | |
|---|---|
| `load_env()` | Loads KEY=VALUE lines from the repo's .env into os.environ (existing variables win). |
| `database_url()` | The read-only connection string (ANALYTICS_DATABASE_URL, or the local development default). |
| `Caller` `class` | The signed-in person: who, in which tenant, with which roles. |
| `tenant_cursor(tenant_id: str)` | Yields a dict-row cursor in a read-only transaction scoped to one tenant by row-level security. |
| `caller_from_session(raw_token: str)` | Resolves a raw session cookie to the caller (None when missing, expired or the user is inactive). |

### apps/analytics/arc_analytics/excel.py

Excel export — GRASP Pure Fabrication: turns KPI rows into a tidy .xlsx workbook (openpyxl).

| | |
|---|---|
| `kpi_workbook(projects: list[dict[str, Any]], employees: list[dict[str, Any]], date…)` | Builds the KPI workbook (sheets: Projekter, Medarbejdere, Om) and returns the .xlsx bytes. |

### apps/analytics/arc_analytics/kpi.py

Key figures — the Python mirror of packages/domain/src/kpi.ts (GRASP Information Expert).

| | |
|---|---|
| `div_round(numerator: int, denominator: int)` | Integer division rounding half away from zero (mirror of divRound in money.ts). |
| `utilisation(billable: int, norm: int)` | Utilisation ("udnyttelsesgrad"): billable hours ÷ norm hours, in basis points. |
| `billable_pct(billable: int, work: int)` | Billable share ("faktureringsgrad"): billable ÷ all registered work hours, in basis points. |
| `effective_rate(invoiced: int, hours: int)` | Effective hourly rate: invoiced øre ÷ invoiced centi-hours, in øre per hour (None without hours). |
| `margin(revenue: int, cost: int)` | Contribution margin ("dækningsbidrag") in øre and margin % ("DB-grad") in basis points. |

### apps/analytics/arc_analytics/main.py

ARCnordic Tid analytics service (D-002, D-021) — FastAPI, read-only.

| | |
|---|---|
| `current_caller(arc_sid: Annotated[str \| None, Cookie()]` | Resolves the signed-in caller from the session cookie; 401 without a session, 403 without a KPI role. |
| `health()` | Liveness probe (no database access). |
| `kpis(caller: CallerDep, p: PeriodDep)` | Project and employee KPIs for the period (hours in centi-hours, money in øre, percentages in basis points). |
| `kpis_excel(caller: CallerDep, p: PeriodDep)` | The KPI report as an Excel download. |
| `get_dataset(name: str, caller: CallerDep, p: PeriodDep)` | A flat dataset for Power BI's Web connector or Excel's Power Query. |

### apps/analytics/arc_analytics/queries.py

Read-only report queries — GRASP Pure Fabrication: SQL that gathers raw numbers; the KPI maths stays in kpi.py.

| | |
|---|---|
| `project_kpis(cur: psycopg.Cursor, date_from: str, date_to: str)` | Per project: hours, billable hours, cost, invoiced revenue and invoiced hours → billable %, rate, margin. |
| `employee_kpis(cur: psycopg.Cursor, date_from: str, date_to: str)` | Per employee: work hours, billable hours, absence hours and billable %. |
| `dataset(cur: psycopg.Cursor, name: str, date_from: str, date_to: str, sees_re…)` | Rows of one flat dataset (see DATASETS); raises KeyError for an unknown name. |

## 10. API-endpoints

103 endpoints under `/api/v1`. "Adgang" er den kapabilitet (fra rollematrixen) eller regel, ruten tjekker; services begrænser derudover, *hvis* data man ser.

| Metode | Sti | Fil | Adgang |
|---|---|---|---|
| GET | /api/v1/admin/settings | routes/admin.ts | settings.read |
| PATCH | /api/v1/admin/approval-mode | routes/admin.ts | settings.manage |
| POST | /api/v1/admin/norm-patterns | routes/admin.ts | settings.manage |
| POST | /api/v1/admin/holiday-calendars/:id/generate | routes/admin.ts | settings.manage |
| POST | /api/v1/admin/holidays | routes/admin.ts | settings.manage |
| DELETE | /api/v1/admin/holidays/:id | routes/admin.ts | settings.manage |
| PATCH | /api/v1/admin/absence-codes/:id | routes/admin.ts | settings.manage |
| GET | /api/v1/admin/employees/:id | routes/admin.ts | settings.read |
| PATCH | /api/v1/admin/employees/:id | routes/admin.ts | settings.manage / settings.read |
| POST | /api/v1/admin/employees/:id/norms | routes/admin.ts | settings.manage |
| GET | /api/v1/approvals/projects | routes/approval.ts | login + regler i service |
| POST | /api/v1/approvals/projects/decide | routes/approval.ts | login + regler i service |
| GET | /api/v1/approvals | routes/approval.ts | login + regler i service |
| POST | /api/v1/approvals/:periodId/approve | routes/approval.ts | login + regler i service |
| POST | /api/v1/approvals/bulk-approve | routes/approval.ts | login + regler i service |
| POST | /api/v1/approvals/:periodId/reject | routes/approval.ts | login + regler i service |
| POST | /api/v1/approvals/:periodId/reopen | routes/approval.ts | login + regler i service |
| GET | /api/v1/timesheet/calendar/:weekStart | routes/calendar.ts | login + regler i service |
| POST | /api/v1/timesheet/calendar/import | routes/calendar.ts | login + regler i service |
| POST | /api/v1/timesheet/calendar/ignore | routes/calendar.ts | login + regler i service |
| POST | /api/v1/import/:kind/preview | routes/data-import.ts | data.import / people.import |
| POST | /api/v1/import/:kind/commit | routes/data-import.ts | data.import / people.import |
| GET | /api/v1/expenses/settings | routes/expenses.ts | login + regler i service |
| GET | /api/v1/expenses | routes/expenses.ts | login + regler i service |
| POST | /api/v1/expenses | routes/expenses.ts | login + regler i service |
| PUT | /api/v1/expenses/:id | routes/expenses.ts | login + regler i service |
| DELETE | /api/v1/expenses/:id | routes/expenses.ts | login + regler i service |
| POST | /api/v1/expenses/submit | routes/expenses.ts | login + regler i service |
| GET | /api/v1/expense-approvals | routes/expenses.ts | login + regler i service |
| POST | /api/v1/expenses/:id/approve | routes/expenses.ts | login + regler i service |
| POST | /api/v1/expenses/:id/reject | routes/expenses.ts | login + regler i service |
| POST | /api/v1/files | routes/expenses.ts | login + regler i service |

| | | | |
|---|---|---|---|
| GET | /api/v1/files/:id | routes/expenses.ts | login + regler i service |
| GET | /api/v1/periods | routes/finance.ts | period.view |
| GET | /api/v1/periods/:month/report | routes/finance.ts | period.view |
| POST | /api/v1/periods/:id/close | routes/finance.ts | period.close |
| POST | /api/v1/periods/:id/reopen | routes/finance.ts | period.close |
| GET | /api/v1/price-lists | routes/finance.ts | login + regler i service |
| POST | /api/v1/price-lists/:id/rates | routes/finance.ts | pricelist.manage |
| DELETE | /api/v1/price-list-rates/:id | routes/finance.ts | pricelist.manage |
| GET | /api/v1/projects/:id/payment-plan | routes/finance.ts | login + regler i service |
| POST | /api/v1/contracts/:id/plan-items | routes/finance.ts | login + regler i service |
| PATCH | /api/v1/plan-items/:id | routes/finance.ts | login + regler i service |
| GET | /api/v1/audit | routes/governance.ts | audit.read |
| GET | /api/v1/audit/entities | routes/governance.ts | audit.read |
| GET | /api/v1/admin/employees/:id/gdpr-export | routes/governance.ts | gdpr.manage |
| GET | /api/v1/admin/employees/:id/erasure | routes/governance.ts | gdpr.manage |
| POST | /api/v1/admin/employees/:id/erase | routes/governance.ts | gdpr.manage |
| GET | /api/v1/admin/retention | routes/governance.ts | gdpr.manage |
| GET | /api/v1/invoicing/basis | routes/invoicing.ts | login + regler i service |
| POST | /api/v1/invoices/drafts | routes/invoicing.ts | login + regler i service |
| POST | /api/v1/invoices/bulk-drafts | routes/invoicing.ts | invoice.prepare_all |
| GET | /api/v1/invoices | routes/invoicing.ts | login + regler i service |
| GET | /api/v1/invoices/:id | routes/invoicing.ts | login + regler i service |
| PATCH | /api/v1/invoices/:id | routes/invoicing.ts | login + regler i service |
| PATCH | /api/v1/invoice-lines/:id | routes/invoicing.ts | login + regler i service |
| POST | /api/v1/invoices/:id/lines | routes/invoicing.ts | login + regler i service |
| DELETE | /api/v1/invoice-lines/:id | routes/invoicing.ts | login + regler i service |
| DELETE | /api/v1/invoices/:id | routes/invoicing.ts | login + regler i service |
| POST | /api/v1/invoices/:id/ready | routes/invoicing.ts | login + regler i service |
| POST | /api/v1/invoices/:id/book | routes/invoicing.ts | invoice.book |
| POST | /api/v1/invoices/:id/credit | routes/invoicing.ts | invoice.credit |
| GET | /api/v1/invoices/:id/html | routes/invoicing.ts | login + regler i service |
| GET | /api/v1/invoices/:id/pdf | routes/invoicing.ts | login + regler i service |
| GET | /api/v1/payroll/periods | routes/payroll.ts | payroll.export |
| GET | /api/v1/payroll/periods/:month | routes/payroll.ts | payroll.export |
| POST | /api/v1/payroll/periods/:month/export | routes/payroll.ts | payroll.export |
| POST | /api/v1/payroll/periods/:month/lock | routes/payroll.ts | payroll.export |
| GET | /api/v1/payroll/periods/:month/file.csv | routes/payroll.ts | payroll.export |

| | | | |
|---|---|---|---|
| GET | /api/v1/payroll/salary-codes | routes/payroll.ts | payroll.setup |
| PATCH | /api/v1/payroll/salary-codes/:id | routes/payroll.ts | payroll.setup |
| PUT | /api/v1/payroll/absence-mapping | routes/payroll.ts | payroll.setup |
| GET | /api/v1/planning | routes/planning.ts | login + regler i service |
| PUT | /api/v1/planning/allocations | routes/planning.ts | login + regler i service |
| GET | /api/v1/planning/competencies | routes/planning.ts | login + regler i service |
| PUT | /api/v1/users/:id/competencies | routes/planning.ts | login + regler i service |
| GET | /api/v1/projects | routes/projects.ts | login + regler i service |
| GET | /api/v1/projects/:id | routes/projects.ts | login + regler i service |
| POST | /api/v1/projects | routes/projects.ts | project.create |
| POST | /api/v1/projects/:id/tasks | routes/projects.ts | login + regler i service |
| PATCH | /api/v1/tasks/:id | routes/projects.ts | login + regler i service |
| GET | /api/v1/customers | routes/projects.ts | login + regler i service |
| POST | /api/v1/customers | routes/projects.ts | login + regler i service |
| GET | /api/v1/me/balances | routes/reports.ts | login + regler i service |
| GET | /api/v1/me/dashboard | routes/reports.ts | login + regler i service |
| GET | /api/v1/reports/budget-actual | routes/reports.ts | login + regler i service |
| GET | /api/v1/reports/time-entries | routes/reports.ts | login + regler i service |
| GET | /api/v1/timesheet/week/:weekStart | routes/timesheet.ts | login + regler i service |
| PUT | /api/v1/timesheet/entries | routes/timesheet.ts | login + regler i service |
| POST | /api/v1/timesheet/week/:weekStart/submit | routes/timesheet.ts | login + regler i service |
| POST | /api/v1/timesheet/week/:weekStart/reopen | routes/timesheet.ts | login + regler i service |
| GET | /api/v1/timesheet/tasks | routes/timesheet.ts | login + regler i service |
| POST | /api/v1/timesheet/favourites | routes/timesheet.ts | login + regler i service |
| DELETE | /api/v1/timesheet/favourites/:taskId | routes/timesheet.ts | login + regler i service |
| GET | /api/v1/timesheet/week/:weekStart/previous-rows | routes/timesheet.ts | login + regler i service |
| GET | /api/v1/me | routes/users.ts | login + regler i service |
| GET | /api/v1/users | routes/users.ts | login + regler i service |
| PUT | /api/v1/users/:id/roles | routes/users.ts | settings.manage |
| GET | /api/v1/auth/providers | auth/routes.ts | offentlig |
| POST | /api/v1/auth/dev-login | auth/routes.ts | offentlig |
| POST | /api/v1/auth/logout | auth/routes.ts | login + regler i service |
| GET | /api/v1/auth/entra/login | auth/routes.ts | offentlig |
| GET | /api/v1/auth/entra/callback | auth/routes.ts | offentlig |

## 11. Datamodel

50 tabeller i PostgreSQL (`packages/db/src/schema.ts`). Alle tabeller med `tenant_id` er beskyttet af row-level security.

| Tabel | Kolonner | Formål |
|---|---|---|
| `tenants` | 9 | |
| `legal_entities` | 8 | |
| `departments` | 9 | |
| `users` | 22 | |
| `user_roles` | 3 | |
| `sessions` | 6 | |
| `audit_log` | 10 | |
| `norm_patterns` | 6 | |
| `employee_norms` | 7 | |
| `holiday_calendars` | 4 | |
| `holidays` | 7 | |
| `absence_codes` | 11 | |
| `customers` | 16 | |
| `contacts` | 7 | |
| `projects` | 17 | |
| `contracts` | 10 | |
| `tasks` | 16 | |
| `project_members` | 3 | |
| `timesheet_periods` | 10 | |
| `timesheet_events` | 7 | |
| `time_entries` | 21 | |
| `favourites` | 3 | |
| `price_groups` | 4 | Rate role of an employee (e.g. |
| `task_types` | 3 | |
| `price_lists` | 7 | |
| `price_list_rates` | 7 | |
| `project_employee_rates` | 6 | |
| `payment_plan_items` | 8 | Fixed-price payment plan / retainer fee schedule. |
| `number_series` | 5 | Gap-free invoice numbering per legal entity (brief §4.10). |
| `invoices` | 27 | |
| `invoice_lines` | 16 | |
| `invoice_line_sources` | 7 | Which time entries an invoice line covers, with registered and invoiced value (write-down analytics). |

| | | |
|---|---|---|
| accounting_periods | 7 | |
| revenue_recognitions | 8 | Snapshot taken when a month is closed: fixed-price revenue recognised to date and the month's delta. |
| integration_connections | 7 | Configured accounting/payroll connections. |
| outbox | 9 | Transactional outbox: events written in the same transaction as the change, delivered by a worker. |
| expense_types | 6 | Expense categories; `vatDeductibleBp` is the share of the VAT the company may deduct (e.g. |
| mileage_rates | 6 | Mileage allowance per year: high rate (øre/km) up to the threshold, low rate above. |
| files | 8 | Uploaded files (receipts). |
| expenses | 26 | An out-of-pocket expense or a mileage claim. |
| allocations | 7 | Planned hours: one employee on one project in one ISO week (decision OQ-28 default: weekly granularity). |
| user_competencies | 3 | Competencies as simple tags per employee (e.g. |
| reminder_log | 6 | E-mail reminders already sent, so the weekly job never sends the same reminder twice. |
| graph_tokens | 4 | Delegated Microsoft Graph access token per user (read-only scope Calendars.Read), AES-GCM encrypted with a key derived from SESSION_SECRET. |
| calendar_imports | 9 | Calendar events (per day) the user has imported into the timesheet or chosen to ignore — shown as handled. |
| salary_codes | 9 | Salary codes ("lønarter") the payroll system knows. |
| pay_periods | 8 | Monthly pay periods: open → exported → locked. |
| payroll_runs | 7 | One generated export file for a pay period. |
| payroll_lines | 9 | Lines of a run; `month` is the month the items are dated in (earlier than the run's month for late lines). |
| user_rights | 4 | Extra rights on top of roles (D-025), e.g. |

# 12. Beslutningslog

| ID | Dato | Beslutning | Status |
|---|---|---|---|
| D-001 | 2026-11-08 | Repository location | accepted |
| D-002 | 2026-11-08 | Languages: TypeScript core, Python analytics service | accepted |
| D-003 | 2026-11-08 | Contracts are child objects of projects | accepted |
| D-004 | 2026-11-08 | Fixed-price completion | accepted |
| D-005 | 2026-11-08 | Hour input formats | accepted |
| D-006 | 2026-11-08 | Approval granularity | accepted |
| D-007 | 2026-11-08 | Visual identity | proposed |
| D-008 | 2026-11-08 | Retainers as a contract type | accepted |
| D-009 | 2026-11-08 | Toolchain | accepted |
| D-010 | 2026-11-08 | Product name in code | accepted |
| D-011 | 2026-11-08 | Backend stack details | accepted |
| D-012 | 2026-11-08 | Multi-tenancy enforcement | accepted |
| D-013 | 2026-11-08 | Business-rule defaults (owner delegated: "just make something") | |
| D-014 | 2026-11-08 | Contract hourly rate in the rate order | accepted |
| D-015 | 2026-11-08 | Invoice numbering and immutability | accepted |
| D-016 | 2026-11-08 | PDF rendering | accepted |
| D-017 | 2026-11-08 | Run locally, no hosting for now | accepted |
| D-018 | 2026-11-08 | Code structure (GRASP) and documentation rules | accepted |
| D-019 | 2026-11-08 | Expenses: approval, VAT, mileage rates, receipts | accepted |
| D-020 | 2026-11-08 | Dashboard, reminders and Outlook calendar import | accepted |
| D-021 | 2026-11-08 | Python analytics service | accepted |
| D-022 | 2026-11-08 | Approval modes | accepted |
| D-023 | 2026-11-08 | Payroll export, first version | accepted |
| D-024 | 2026-11-08 | Data import | accepted |
| D-025 | 2026-11-09 | Access policy and segregation of duties | accepted |
| D-026 | 2026-11-09 | GDPR | accepted |
| D-027 | 2026-11-09 | Operations | proposed |

Fuld tekst med begrundelser: `docs/decisions.md`.

# 13. Kendte begrænsninger og anbefalinger

### Kræver beslutning hos ejeren

- **Hosting (OQ-04)** — forslag i `docs/operations.md`; indtil da kun lokal test med opdigtede data.
- **Flyt arbejdskopien ud af OneDrive (OQ-01)** — trin-for-trin i `docs/operations.md` §4.
- **Åbne fagspørgsmål** — flexloft, ferieoptjening, kilometersatser og moms (OQ-36), nummerserier, lønudbyder (OQ-08), offentlige kunder/OIOUBL (OQ-32), opbevaringsfrist og regnskabsår (OQ-39). Systemet kører på standardvalg indtil da.

### Bevidste forenklinger

- Løneksport er generisk CSV; adaptere til Danløn/DataLøn/Zenegy kommer, når udbyderen er valgt.
- Persondata-udtræk er én JSON-fil (spec nævner ZIP med JSON+CSV).
- Revisionssporet er append-only men endnu ikke hash-kædet (spec R2 foreslår kæde for manipulationssikring).
- Kalender-tokens fornyes ikke automatisk (ingen refresh-tokens gemmes) — brugeren logger ind med Microsoft igen.
- Bogføringsdata ud over 5 år rapporteres, men slettes ikke automatisk.

### Gode næste skridt

- Entra ID med automatisk oprettelse og gruppe → rolle (OQ-10), så roller ikke vedligeholdes to steder.
- Projektskabeloner med faser fra ydelsesbeskrivelsen (OQ-23).
- Mobil/PWA til tidsregistrering på byggepladsen.
- API-tokens (spec R10), så Power BI kan opdatere automatisk.